CS-BAPR: Formal OOD Extrapolation Bounds for Reinforcement Learning via Neural Arithmetic Units*

Yifan Zhang† Liang Zheng‡

Abstract

Standard reinforcement learning policies trained on bounded state regions fail catastrophically when deployed in out-of-distribution (OOD) regimes. A key yet underexplored source of this failure is *architectural*: ReLU-based policy networks exhibit derivative discontinuities that permit unbounded error growth in extrapolation regions—a structural deficiency that no amount of data can remedy.

We propose **CS-BAPR**, a framework that extends BAPR [1] along two axes: (i) a set of *six training-stabilization techniques*—four CS-BAPR base fixes (min-Q ensemble target, running-EMA reward normalization, entropy-temperature floor, and disabling the BAPR-inherited behavioural-cloning anchor for constrained heads) plus two ports of BAPR v10/v14 (the v10 BOCD-warmup-and-penalty-scale recipe, and the v14 rollout-based one-sided surprise reform; Section 5.3 details all six)—that jointly prevent late-training collapse of SAC ensembles in sparse-reward transit control; and (ii) a choice of *actor-head architecture*—Neural Arithmetic Unit (NAU/NMU), Kolmogorov-Arnold Network (KAN), or plain MLP-with-tanh—with the first two providing finite or zero derivative Lipschitz constants, and thus the architectural foothold for a polynomial OOD bound. We present these as a family of closely-related methods rather than claiming a single architecture dominates; our ablation on a 12-line SUMO-calibrated transit benchmark shows that the six training-stabilization fixes account for the bulk of the improvement over the BAPR baseline, while the architectural choices trade off ID quality, OOD stability, and run-to-run variance in non-uniform ways.

Our main theoretical result is the **OOD Extrapolation Stability Bound**: if the policy achieves base accuracy δ on the training domain, derivative error ε at the training boundary, and derivative Lipschitz constant L, then the OOD prediction error at distance $\|d\|$ from a reference point satisfies $\|\pi(x_{\text{ood}}) - \pi_{\text{ref}}(x_{\text{ood}})\| \leq \delta + \varepsilon\|d\| + \frac{L}{2}\|d\|^2$, where π_{ref} is any differentiable reference function. We further prove that ReLU lacks any finite derivative-Lipschitz constant, establishing a sharp architectural boundary for this specific certificate—ReLU cannot instantiate the derivative-Lipschitz bound, while NAU gives $L = 0$ and NMU gives $L = 2|c|$. The CS-BAPR Bellman operator preserves BAPR's γ-contraction guarantee. We provide detailed mathematical proofs and formalization-oriented theorem statements for all results.

As *optional framework extensions*, we formalize a SINDy-based Jacobian Consistency pipeline and an IRM causal-filtering step that can tighten the ε term when the environment admits sparse, invariant dynamics *and* the optimal policy is aligned with the identified dynamics structure. These extensions are *not* required for the core bound, and whether they empirically help is domain-dependent; they are not activated on our primary benchmark (Section 3). On the LQR auxiliary benchmark, the *action-aware* SINDy variant (jointly identifying $f(x, u) = Ax + Bu$) yields a 66−97% improvement over the no-SINDy ablation at mid-OOD (m=5−10× initial-state amplitude); this is the first positive empirical case for the SINDy extension and validates the architectural prediction that closed-loop alignment is the operative requirement.

We evaluate on a SUMO-calibrated 12-line transit holding benchmark with demand-magnitude OOD (up to 50× in-distribution flow), an abrupt-burst shift protocol (single step change at

*Code available at https://github.com/erzhu419/CS-BAPR. The acronym "CS-BAPR" is retained for continuity with the authors' prior work and repository; the primary mechanism in this paper is the NAU/NMU actor head, while the causal-symbolic components (SINDy + IRM) are presented as optional framework extensions with a deliberately narrower empirical scope (see Section 3).

†Central South University, erzhu419@gmail.com, 204201048@csu.edu.cn

‡Central South University, zhengliang@csu.edu.cn

t=3600s), and a within-episode piecewise-constant oscillation protocol (multiple switches per episode, peak 20−50×). Across all three protocols, the CS-BAPR variants improve over the clean unstabilized BAPR baseline, with the training-stabilized MLP variant providing the most consistent behavior at the extreme end (20×−50×), the NAU variant providing the best static mid-range OOD point (5×) at the cost of higher run-to-run variance, and the KAN variant providing the best in-distribution accuracy.

Keywords: out-of-distribution extrapolation; symbolic regression; neural arithmetic units; causal invariance; formal verification; robust reinforcement learning.

1 Introduction

Reinforcement learning (RL) has achieved remarkable success in domains where training and deployment conditions are well-aligned [3]. However, real-world applications frequently require policies to operate in regimes far beyond their training distribution—a challenge we term *magnitude extrapolation*. Unlike the interpolation-based generalization studied in domain randomization [5] or the regime-switch adaptation addressed by BAPR [1], magnitude extrapolation demands that the agent's output scale correctly to input magnitudes never encountered during training.

Definition 1.1 (Extrapolation Failure)**.** *Let $\pi_\theta : \mathcal{S} \to \mathcal{A}$ be a policy trained on domain $D_{train} \subset \mathcal{S}$, and let $\pi_{ref} : \mathcal{S} \to \mathcal{A}$ be any differentiable reference function used to characterize admissible extrapolation (for example, the policy's own linearization at the training boundary, or a symbolically identified reference when one is available; see Assumption 5.2). The policy suffers from* extrapolation failure *if there exists a distance threshold $d_0 > 0$ such that for x_{ood} with $\|x_{ood} - D_{train}\| > d_0$:*

(i) ***Error explosion:*** *$\|\pi_\theta(x_{ood}) - \pi_{ref}(x_{ood})\|$ grows super-polynomially with distance;*

(ii) ***Gradient saturation:*** *$\|\nabla_x \pi_\theta(x_{ood})\| \to 0$ or exhibits discontinuous jumps, preventing meaningful response scaling.*

The bound of Theorem 5.39 rules out (i) (giving polynomial rather than super-polynomial growth), and the NAU architecture (Section 5.2) rules out (ii) (giving a constant derivative rather than saturating/jumping).

This failure mode arises from two orthogonal sources: (a) *training-time instability* that pushes the actor toward poorly-calibrated Q-value estimates and eventually toward degenerate policies under the SAC objective, and (b) *architectural* properties of the function class—specifically, the inability of ReLU-based heads to admit polynomial OOD bounds even in principle. CS-BAPR addresses both:

1. **Training-stabilization fixes.** Extending BAPR's RE-SAC backbone, we introduce four interacting fixes that together keep the ensemble critic and the actor from late-run collapse on the transit benchmark: (i) a *min-Q ensemble target* to suppress over-optimism during the Q-backup, (ii) *running-EMA reward normalization* (divide by EMA of batch std, no mean shift) to keep TD magnitudes scale-invariant under late-training reward drift, (iii) an *entropy-temperature floor* ($\alpha \geq 0.01$) to prevent entropy collapse, and (iv) disabling the BAPR-inherited behavioural-cloning anchor ($\beta_{\text{bc}} = 0$) for the constrained-head variants, where the architecture itself already bounds policy drift. These fixes are architecture-agnostic and, as our ablation shows, account for the majority of the improvement over the BAPR baseline under demand OOD.

2. **Architecturally-varied actor head.** CS-BAPR admits three interchangeable head choices for the actor, each with a different OOD profile:

   - *NAU/NMU* [12]: constant derivative ($L = 0$ at the head), giving the *tightest* instance of our polynomial OOD bound (Theorem 5.39).
   - *KAN* [13]: learnable B-spline edges with smooth (bounded-Lipschitz) derivatives, giving a bounded but nonzero L via the spline grid.

- *Plain MLP with tanh head*: simplest baseline; supports the bound under standard spectral-normalization of the feature extractor.

For the ReLU alternative, we prove that no finite derivative-Lipschitz constant L exists, so ReLU policies cannot instantiate the particular certificate $\delta + \varepsilon\|d\| + \frac{L}{2}\|d\|^2$ used in this paper (Theorem 5.13). This does not rule out every possible ReLU extrapolation analysis; it rules out this derivative-Lipschitz route. In our experiments, the architecture choices trade off ID accuracy, mid-OOD stability, and run-to-run variance in non-uniform ways: we present them as a *family* of closely-related methods rather than asserting a single winner.

3. **(Optional) Symbolic extensions.** We also formalize a SINDy-based Jacobian Consistency pipeline [15] and an Invariant Risk Minimization (IRM) [18] filtering step for symbolic formula candidates. These tighten the ε term in the OOD bound when the environment admits sparse, invariant dynamics *and* the optimal policy is structurally aligned with those dynamics. Our primary transit benchmark does not satisfy the alignment condition—transit holding is a counteraction task, not a tracking task—so we do not activate the symbolic extensions on it. They are retained as optional framework components for future benchmarks in the tracking regime (see Section 3).

Building upon the robust ensemble framework of RE-SAC [2] and the Bayesian change-point adaptation of BAPR [1], our contributions are:

- **CS-BAPR family of methods.** A practical training recipe (six stabilization fixes: four base CS-BAPR fixes plus two ports of BAPR v10/v14) combined with four evaluated actor-head variants: ReLU-MLP with fixes, NAU, smooth MLP-with-tanh, and KAN. All CS-BAPR variants improve over the clean unstabilized BAPR baseline on the 12-line SUMO-calibrated transit benchmark. The strongest variant depends on the OOD regime: `csbapr-kan` is best in-distribution and near-OOD (m=1−2), `csbapr` (NAU) is best at the static m=5× point, `csbapr-no-nau` is best at static m=10−20× and on the oscillating schedules, and `csbapr-tanh` is best at static m=50×.

- **OOD Extrapolation Stability Bound (Theorem 5.39).** Under boundary Jacobian Consistency with a chosen reference (ε) and bounded derivative Lipschitz constant ($L < \infty$), the OOD error satisfies $\|\pi(x_{\text{ood}}) - \pi_{\text{ref}}(x_{\text{ood}})\| \le \delta + \varepsilon\|d\| + \frac{L}{2}\|d\|^2$. The quadratic coefficient $L/2$ is controlled by architecture: $L = 0$ for NAU, $L = 2|c|$ for NMU, bounded for KAN and spectrally-normalized smooth MLPs, and absent for ReLU under this certificate.

- **ReLU derivative-Lipschitz impossibility (Theorem 5.13).** ReLU lacks any finite derivative-Lipschitz constant, so it cannot instantiate the certificate above. This is the sharp *structural* boundary between the bound-compatible heads and the BAPR ReLU baseline for this proof route.

- **CS-BAPR Bellman Contraction (Theorem 5.25).** The CS-BAPR operator, extending BAPR with an (optional, frozen) symbolic consistency penalty Γ_{sym}, is a γ-contraction in L_∞. The contraction holds whether or not Γ_{sym} is enabled, as long as it (and Γ_{epi}, κ) are frozen during each backup, and does not depend on the choice of actor head.

- **Formal symbolic extensions (empirically optional).** Mathematical formalization of the SINDy-to-OOD pipeline (Theorem 5.19) and an IRM causal-filtering statement (Theorem 5.22). These are off on the transit benchmark; they are retained as framework-level components for future applications where the policy-dynamics alignment condition holds.

- **Proof transparency.** We give complete paper proofs and explicit formalization tags for the key definitions, assumptions, and theorem dependencies, using standard tools such as the convex Mean Value Theorem and Grönwall-style fencing arguments.

2 Related work

2.1 OOD generalization in RL

Generalization in RL has been studied through domain randomization [5], distributional robustness [9, 8], and epistemic uncertainty quantification [10]. These approaches provide *interpolation* guarantees within the convex hull of training environments but do not address *extrapolation* to magnitudes beyond the training support. Robust MDPs [6, 7] hedge against worst-case perturbations but with *static* uncertainty sets that become vacuous far from the training distribution. CS-BAPR instead leverages *architectural* structure (NAU's constant-derivative property) to control how much policy response is allowed to drift per unit OOD distance, with an optional symbolic-reference extension for domains where such a reference is meaningful.

2.2 Neural arithmetic units

Neural Arithmetic Logic Units (NALU) [11] introduced the idea of discrete-weight arithmetic layers for numerical extrapolation. Neural Arithmetic Units (NAU) and Neural Multiplication Units (NMU) [12] improved stability by separating addition and multiplication paths and using regularization-based weight discretization. These architectures have been applied to function learning tasks [14] but not to RL policy networks or formal OOD bounds. CS-BAPR provides derivative-Lipschitz guarantees for NAU/NMU architectures and proves that ReLU lacks this property.

2.3 Symbolic regression and SINDy

Sparse Identification of Nonlinear Dynamics (SINDy) [15] discovers governing equations from data using sparse regression over a candidate function library. SINDy-RL [16] integrates SINDy with model-based RL for interpretable dynamics models. AI Feynman [17] uses physics-inspired symbolic regression for equation discovery. CS-BAPR formalizes the SINDy identification process and proves that successful identification implies Jacobian Consistency, which in turn implies polynomial OOD bounds.

2.4 Causal RL and invariant risk minimization

Invariant Risk Minimization (IRM) [18] seeks predictors whose optimal classifier is invariant across training environments, targeting causal rather than spurious correlations. Causal representation learning [19] and causal RL surveys [21] provide the theoretical foundations. CS-BAPR applies IRM not to the policy directly but to the *symbolic formula selection* stage, filtering SINDy candidates to retain only causally invariant physical laws.

2.5 Physics-informed ML and ReLU failures

Recent work has identified systematic failures of ReLU activations in physics-informed machine learning [22], showing that ReLU's piecewise linearity prevents accurate learning of smooth physical solutions. Our ReLU Impossibility Theorem (Theorem 5.13) provides the first *formal proof* that this failure extends to derivative Lipschitz continuity—a necessary condition for polynomial OOD bounds within our framework. We emphasize that the impossibility is specific to bounds requiring Lipschitz-continuous derivatives; alternative frameworks (e.g., piecewise-linear extrapolation analysis) may yield different guarantees for ReLU networks.

2.6 Formal methods in RL

Formal verification of RL algorithms remains rare. Our prior work established mechanized contraction proofs for RE-SAC [2] and BAPR [1]. CS-BAPR follows the same formal-methods style by making the OOD extrapolation assumptions, architectural impossibility result, and optional SINDy/IRM framework components explicit enough to audit theorem-by-theorem. We emphasize that formal

proof is a *design audit tool*, not a replacement for experimental validation: the proofs tell us the bounds hold *given* their assumptions; whether the assumptions hold in a specific domain is an empirical question (Section 3).

3 Scope and applicability

Before introducing the method, we delineate the regimes in which CS-BAPR's components are expected to help. The framework has one *primary* mechanism that applies broadly, and two *optional* symbolic mechanisms whose usefulness is domain-specific.

Primary mechanism (NAU head + training stabilization): always applicable. The NAU/NMU output head provides a finite derivative Lipschitz constant L regardless of domain, and the OOD bound $\|\pi(x_{\text{ood}}) - \pi_{\text{ref}}(x_{\text{ood}})\| \leq \delta + \varepsilon\|d\| + \frac{L}{2}\|d\|^2$ holds for any differentiable reference π_{ref}—including the trivial choice $\pi_{\text{ref}}(x) = \pi(x_0) + D\pi(x_0)(x - x_0)$ (the policy's own linear extrapolation from the training boundary). In this NAU-only configuration, ε is simply the policy's boundary fitting error, and the bound controls *how fast* the policy's output can drift per unit OOD distance. This is the guarantee our primary experiments validate.

Optional SINDy extension: applicable when the optimal policy tracks identified dynamics. The SINDy-based symbolic extension sharpens ε by replacing the trivial reference with a symbolically identified one. It is expected to help when:

(i) the environment dynamics $\dot{x} = f(x, u)$ admit a sparse representation in a chosen basis library (Assumption 5.3), *and*

(ii) the optimal policy is itself expressible, or approximately expressible, in the same basis—equivalently, the task is of the "policy should scale with physics" type (e.g., ballistic correction, classical tracking control, energy shaping).

It is *not* expected to help, and can hurt, when the optimal policy's job is to *oppose* the identified dynamics (e.g., bus holding control, where the controller's purpose is to counteract demand-driven headway spreading rather than follow its gradient). Our main transit benchmark is an instance of this latter regime, and an earlier variant of CS-BAPR that included Γ_{sym} on this benchmark degraded rather than improved OOD performance. We therefore report the NAU+stabilization pipeline as the primary experimental configuration, with the symbolic extensions presented as formalized but deliberately not activated on this benchmark.

Optional IRM extension: applicable when SINDy is active and multiple environments are available. The IRM filtering step is a sub-extension of the SINDy pipeline: it refines the symbolic reference across multiple training environments to isolate invariant structure. It inherits the same applicability conditions as the SINDy extension, plus the standard IRM requirement of $K \geq 3$ environments with shared invariant structure.

Takeaway. The paper's empirically defended claim is on the primary mechanism. The symbolic extensions are formalized (and formal) so that the framework is complete for domains where they apply, but we do not empirically claim they improve transit holding control, because the framework itself predicts they should not for that type of task.

4 Preliminaries

4.1 Maximum entropy RL

Following SAC [3], the agent maximizes the entropy-augmented objective:

$$J(\pi) = \mathbb{E}_{\pi,P}\left[\sum_{t=0}^{\infty} \gamma^t \left(R(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot|s_t))\right)\right], \tag{1}$$

where $\mathcal{H}(\pi(\cdot|s_t))$ is the Shannon entropy and α is the temperature parameter.

4.2 BAPR framework

CS-BAPR builds upon the BAPR framework [1], which integrates Bayesian Online Change Detection (BOCD) [4] with robust ensemble RL for piecewise-stationary environments. The BAPR operator is a belief-weighted mixture of mode-conditional Bellman operators:

$$\mathcal{T}_\rho^{BAPR} Q(s,a) = \sum_{m \in \mathcal{M}} \rho(m) \cdot \mathcal{T}_m Q(s,a), \tag{2}$$

where ρ is a frozen belief distribution over modes derived from the BOCD posterior. Each mode-conditional operator $\mathcal{T}_m$ inherits RE-SAC's disentangled aleatoric (κ) and epistemic (Γ_{epi}) penalties [2]. The operator is a γ-contraction when all penalties and beliefs are frozen during each backup [1].

5 Methodology

5.1 The OOD extrapolation problem

Let $D_{\text{train}} \subset \mathcal{S}$ be the training domain and $x_0 \in D_{\text{train}}$ a reference point. For an OOD test point $x_{\text{ood}} \notin D_{\text{train}}$, we seek to bound $\|\pi(x_{\text{ood}}) - \pi_{\text{ref}}(x_{\text{ood}})\|$ as a function of the distance $d = x_{\text{ood}} - x_0$, where π_{ref} is a differentiable reference chosen as specified in Assumption 5.2.

Remark 5.1 (Choice of reference)**.** *The bound is stated against a generic π_{ref}; its utility depends on the choice:*

- ***NAU-only (primary, always applicable):*** *$\pi_{ref}(x) = \pi(x_0) + D\pi(x_0)(x - x_0)$. Then $\varepsilon = 0$ at x_0 by construction, and the bound becomes $\|\pi(x_{ood}) - \pi_{ref}(x_{ood})\| \leq \delta + \frac{L}{2}\|d\|^2$, purely a statement about how far the policy's output drifts from its own linear extrapolation—controlled only by L. This is the guarantee our empirical results validate.*
- ***Symbolic (optional):*** *$\pi_{ref} = f_{sym}$, a SINDy-identified symbolic reference; then ε is the Jacobian Consistency error between π and f_{sym}. This tightens the bound when f_{sym} is an accurate stand-in for an aligned optimal policy, but we do not claim this regime for the transit benchmark (Section 3).*

In what follows, formulas in the optional SINDy subsections continue to use the symbol f_{real} (or f_{sym}) in the sense of "identified dynamics / symbolic reference"; those formulas can be read with $\pi_{ref} := f_{sym}$ when the extension is active.

We state three structural assumptions under which the CS-BAPR framework operates (one mandatory, two active only when the SINDy extension is enabled):

Assumption 5.2 (Reference function)**.** *A differentiable reference function $\pi_{ref} : \mathcal{S} \to \mathcal{A}$ is chosen, against which the policy is regularized (or toward which the policy is compared for bound purposes). Valid choices include: (a)* the policy's own linearization at the training boundary, *$\pi_{ref}(x) = \pi(x_0) + D\pi(x_0)(x - x_0)$—available for free and used in the NAU-only configuration; (b)* a SINDy-identified symbolic formula *(Assumption 5.3, optional)—available when the environment admits sparse dynamics* and *the optimal policy tracks them. The bound in Section 5.8 is stated generically against π_{ref}; the specific choice determines which of ε, δ, M are controllable by additional mechanisms.*

Assumption 5.3 (SINDy Identifiability — *for the optional symbolic extension only*)**.** If *the SINDy extension is enabled, the optimal response function* $f_{real} : \mathcal{S} \to \mathcal{A}$ *admits a sparse representation in the chosen basis library* $\{\varphi_1, \ldots, \varphi_n\}$*, such that SINDy can identify it with derivative accuracy* $\varepsilon_1 < \varepsilon_{threshold}$ *on the training domain. This assumption is* not *required for the primary NAU-only bound.*

Assumption 5.4 (Smooth Reference — *for the optional symbolic extension only*)**.** If *the SINDy extension is enabled, the reference* π_{ref} *(typically* f_{real}*) is differentiable on the convex hull of* $D_{train} \cup \{x_{ood}\}$*, and its Fréchet derivative is* M*-Lipschitz:* $\|D\pi_{ref}(x_1) - D\pi_{ref}(x_2)\| \leq M\|x_1 - x_2\|$ *for some finite* $M \geq 0$*. This holds for physical systems governed by smooth differential equations with bounded second derivatives (e.g., rigid-body dynamics, fluid mechanics, ballistic trajectories), but excludes environments with discontinuous transitions (e.g., contact-rich manipulation with Coulomb friction). When the NAU-only linear reference is used instead,* $M = 0$ *trivially.*

Assumption 5.5 (Bounded Feature Extractor)**.** *The feature extraction layers of the actor network have a finite Lipschitz constant* $L_{feat} < \infty$*, achievable via spectral normalization or weight clamping. Combined with the NAU/NMU output head's derivative Lipschitz constant* L_{head} *(0 for NAU,* $2|c|$ *for NMU), the composed actor has effective derivative Lipschitz constant* $L_{pol} = L_{feat} \cdot L_{head} < \infty$*. (Formal tag:* `IsFunctionLipschitz` *for the feature extractor,* `HasArithmeticInductiveBias` *for the output head.)*

Remark 5.6 (Elimination of pointwise OOD Jacobian Consistency assumption)**.** *An earlier version of this work required a fourth assumption: pointwise Jacobian Consistency along the entire OOD line segment. This assumption was problematic because training only enforces JC on the replay buffer distribution, not in OOD regions.*

We now derive *the OOD derivative error bound from the three assumptions above, without assuming anything about OOD-segment Jacobian alignment. The key result (Formal tag:* `deriv_error_propagation_nD`*, Part X) shows that for any point* x *(including OOD):*

$$\|D\pi(x) - Df_{real}(x)\| \leq \underbrace{\varepsilon}_{\text{training JC}} + \underbrace{(L_{pol} + M)}_{\text{architecture + physics}} \cdot \|x - x_0\|, \tag{3}$$

where the three terms come from:

(i) ε*: Jacobian Consistency at the training boundary* x_0 *(from the training loss* $\mathcal{L}_{AC}$*);*

(ii) $L_{pol} \cdot \|x - x_0\|$*: how much* π*'s derivative drifts from* x_0 *(bounded by NAU/NMU architecture, Assumption 5.5);*

(iii) $M \cdot \|x - x_0\|$*: how much* f_{real}*'s derivative drifts from* x_0 *(bounded by physics smoothness, Assumption 5.4).*

This is a strict improvement: Eq. (3) is derived *from training-time quantities and architectural properties, with no OOD-segment assumptions. The generalization from training boundary to population is handled by a standard PAC bound (Formal tag:* `jc_generalization_to_ood`*, Part XIII):* $\varepsilon_{pop} \leq \varepsilon_{emp} + O(B/\sqrt{N})$*.*

We introduce two key predicates, formalized in formal:

Definition 5.7 (Jacobian Consistency)**.** *A function* $\pi : E \to F$ *is* ε-Jacobian Consistent *with a symbolic formula* $f_{sym} : E \to F$ *on domain* D *if:*

$$\forall x \in D, \quad \|D\pi(x) - Df_{sym}(x)\| \leq \varepsilon, \tag{4}$$

where D *denotes the Fréchet derivative. (Formal tag:* `IsJacobianConsistent_nD`*.)*

Definition 5.8 (Arithmetic Inductive Bias)**.** *A function* $\pi : E \to F$ *has* L-Arithmetic Inductive Bias *if its Fréchet derivative is* L*-Lipschitz:*

$$\forall x_1, x_2, \quad \|D\pi(x_1) - D\pi(x_2)\| \leq L \cdot \|x_1 - x_2\|. \tag{5}$$

(Formal tag: `HasArithmeticInductiveBias_nD`*.)*

5.2 Pillar I (primary): NAU/NMU arithmetic bias

The primary mechanism replaces the policy's output layers with Neural Arithmetic Units, ensuring finite derivative Lipschitz constants. This is the only pillar that is always active in our experiments.

NAU layer. A Neural Arithmetic Unit [12] computes $y = \sum_j w_j x_j$ with weights constrained to $\{-1, 0, 1\}$ (realized via clamping and regularization). Since the output is linear in the input, the derivative $\partial y/\partial x_i = w_i$ is *constant*—independent of the input value.

Theorem 5.9 (NAU has zero-Lipschitz derivatives)**.** *For any integer weight $w \in \mathbb{Z}$, the function $f(x) = wx$ has derivative-Lipschitz constant* 0*.*

Proof. $f'(x) = w$ for all x, so $|f'(x_1) - f'(x_2)| = |w - w| = 0 \leq 0 \cdot |x_1 - x_2|$. □

For multi-dimensional inputs, the NAU derivative constancy extends naturally:

Theorem 5.10 (NAU partial derivatives are constant)**.** *For an NAU layer with integer weights $w_1, \ldots, w_n \in \mathbb{Z}$ computing $y = \sum_j w_j x_j$, the partial derivative $\partial y/\partial x_i = w_i$ is* constant *everywhere—independent of the input vector. This guarantees that the derivative does not drift in OOD regions.*

NMU layer. A Neural Multiplication Unit [11] computes $f(x) = c \cdot x^2$ with learnable coefficient c. Its derivative $f'(x) = 2cx$ is Lipschitz with constant $L = 2|c|$.

Theorem 5.11 (NMU has bounded-Lipschitz derivatives)**.** *For any coefficient $c \in \mathbb{R}$, the map $x \mapsto cx^2$ has derivative-Lipschitz constant $2|c|$.*

Proof. $f'(x) = 2cx$, so $|f'(x_1) - f'(x_2)| = |2cx_1 - 2cx_2| = 2|c| \cdot |x_1 - x_2|$. □

Corollary 5.12 (NAU ∘ NMU composition)**.** *For weights $w, c \in \mathbb{R}$, HasArithmeticInductiveBias($x \mapsto wc \cdot x^2, 2|wc|$). (Formal tag: `nau_nmu_composition_bias`.)*

ReLU impossibility. In contrast, we prove that ReLU networks cannot satisfy any polynomial OOD bound:

Theorem 5.13 (ReLU derivative is not Lipschitz)**.** $\neg\exists L \in \mathbb{R}$, *HasArithmeticInductiveBias($ReLU, L$). (Formal tag: `relu_derivative_not_lipschitz`.)*

Proof sketch. $\mathrm{ReLU}(x) = \max(0, x)$ has derivative 0 for $x < 0$ and 1 for $x > 0$. For any purported L:

- If $L \leq 0$: take $x_1 = -1, x_2 = 1$. Then $|0 - 1| = 1 \leq L \cdot 2 \leq 0$, contradiction.
- If $L > 0$: take $x_1 = -(4L)^{-1}, x_2 = (4L)^{-1}$. Then $|0-1| = 1 \leq L \cdot 2(4L)^{-1} = 1/2$, contradiction.

The proof is written out by explicitly comparing one-sided derivatives around the kink at the origin. □

Remark 5.14 (Architectural hierarchy for extrapolation)**.** *The formal results establish a sharp hierarchy:*

Architecture	L	***OOD bound***	***formal tag***
NAU $(w \cdot x)$	0	δ *(constant)*	`nau_has_arithmetic_bias_1d`
NMU $(c \cdot x^2)$	$2\|c\|$	$\delta + \varepsilon\|d\| + \|c\|\|d\|^2$	`nmu_has_bounded_arithmetic_bias`
ReLU $(\max(0, x))$	$\nexists$	*unbounded*	`relu_derivative_not_lipschitz`

Remark 5.15 (Theory-practice gap: composed architecture)**.** *The formal proofs establish Lipschitz derivative constants for* isolated *NAU ($L = 0$) and NMU ($L = 2|c|$) layers. The actual actor is a composition $\pi = \tanh \circ (\alpha \cdot NAU + (1-\alpha) \cdot NMU) \circ FeatureNet$. The composed constant satisfies $L_{pol} \leq L_{feat} \cdot 2(1-\alpha)|c| \cdot 1 < \infty$ (Assumption 5.5), preserving the quadratic OOD bound.*

Two Lipschitz concepts. *This analysis crucially distinguishes:*

- Function-Lipschitz*: $|f(x_1) - f(x_2)| \leq K|x_1 - x_2|$ — satisfied by LeakyReLU ($K = 1$), required for the feature extractor.*

- Derivative-Lipschitz*:* $|f'(x_1) - f'(x_2)| \leq L|x_1 - x_2|$ *— satisfied by NAU* ($L = 0$) *and NMU* ($L = 2|c|$)*, but* not *by ReLU or LeakyReLU (derivative discontinuity at* $x = 0$*).*

The composition rule (Formal tag: `composed_deriv_lipschitz_simple`*, Part XI) requires function-Lipschitz for the inner function (feature extractor) and derivative-Lipschitz for the outer function (NAU/NMU head). LeakyReLU in hidden layers is acceptable because only function-Lipschitz is needed there; the OOD bound's derivative-Lipschitz requirement applies only to the output head. See Appendix F for the full* L_{pol} *derivation.*

Comparison with smooth activations. The ReLU impossibility is not an impossibility for *all* alternatives: smooth activations such as GELU, Softplus, and tanh have bounded second derivatives and therefore finite derivative Lipschitz constants. The full picture is therefore a four-way comparison—NAU, NMU, smooth-activation (GELU/Softplus/tanh), and ReLU—of which our Theorem 5.13 only rules out the last. We compare against three of the four heads in Section 6: NAU, KAN (a smooth-activation alternative based on learnable B-splines), and plain MLP-with-tanh (`csbapr-no-nau`); for the latter two the bound holds with finite L. Two reasons motivate NAU over the smooth alternatives in our experiments:

(i) *Constant vs. bounded derivative.* NAU's derivative is not merely Lipschitz (L finite), it is *constant* ($L = 0$). For a state-independent action scaling task (e.g., "hold time proportional to headway deviation"), NAU extrapolates linearly with zero quadratic-term cost; smooth activations pay an $L \cdot d^2/2$ penalty that is bounded but nonzero.

(ii) *Architecturally inspectable* L*.* NAU weights are clamped to $\{-1, 0, 1\}$, so L is computed directly from the weight matrix; for a smooth activation, L depends on the learned weight scale and is not directly inspectable without a separate spectral analysis.

We deliberately do *not* include a single GELU or Softplus baseline in the main experiments because plain MLP-with-tanh (the tanh-Gaussian SAC parameterization, exactly the Group A backbone in `csbapr-no-nau`) is the most widely-used smooth-activation default in continuous-control RL. Table 2 shows that `csbapr-no-nau` is competitive with `csbapr` at extreme OOD, validating that NAU's advantage over smooth alternatives is task-dependent rather than universal: NAU dominates when the optimal policy is approximately scale-linear-in-state (the bus benchmark's mid-OOD regime); smooth activations are at parity at extreme OOD, where action saturation dominates regardless of head Lipschitz constant; and they outperform NAU in tasks whose optimal policy must structurally re-tune (Hopper-wind, Section 6.5).

5.3 Training stabilization for the NAU actor

NAU's constrained $\{-1, 0, 1\}$ weight space is less forgiving than an unconstrained MLP: vanilla SAC training on NAU policies is prone to late-stage collapse as the actor's output range shrinks under the architectural clamping, and the BOCD-driven adaptive conservatism inherited from BAPR makes early training even more brittle. We find six minimal architecture-agnostic fixes to the BAPR training loop that jointly recover stable convergence without modifying the core algorithm or the architectural guarantee. The first four are CS-BAPR additions; the last two are ports of fixes from BAPR v10/v14 [1], validated to also help inside the CS-BAPR family.

Group A — base CS-BAPR fixes (independent of NAU choice).

(A1) **Min-Q ensemble target.** In BAPR's Q-ensemble backup, replace the per-critic target with an elementwise minimum across the ensemble: $y_{\text{target}} = \min_e Q_{\bar{\theta}_e}(s', a')$. Prevents coordinated Q-overestimation from driving the actor into saturated weight configurations. The min operation does not affect contraction (the operator remains γ-contractive; Section 5.6).

(A2) **Running-EMA reward normalization (divide-only).** Rewards are divided by the running standard deviation of batch rewards, updated with EMA coefficient 0.99: $\sigma^2_{t+1} = 0.99\,\sigma^2_t +$

0.01 $\mathrm{Var}(r_{\mathrm{batch}})$. No mean shift, preserving sign. Critical for NAU because the clamped weight space is sensitive to reward-magnitude drift across long training runs.

(A3) **Entropy-temperature floor.** The learned SAC temperature α is floored at $\alpha_{\min} = 0.01$: $\alpha = \max(\exp(\log \alpha), \alpha_{\min})$. Prevents the entropy collapse $\alpha \to 0$ that would lose exploration capacity for the remainder of training.

(A4) **Behavioral-cloning anchor disabled for constrained heads.** The BAPR pipeline includes an optional anchor $\beta_{\mathrm{bc}} \|\mu_\pi - a\|^2$. For NAU and KAN we set $\beta_{\mathrm{bc}} = 0$: the architectural constraint already prevents policy drift, and the BC term over-regularizes the already-constrained output. Plain-MLP variants (`csbapr-no-nau`) keep $\beta_{\mathrm{bc}} > 0$.

Group B — ported from BAPR v10/v14.

(B5) **BAPR v10 BOCD-stability recipe (P0+P1, applied jointly).** Two coupled knobs proposed together in BAPR v10. *P0 — mechanism warmup*: for the first `bapr_warmup_iters`= 100 training iterations, force the BOCD-derived weighted lambda λ_w to zero ($\lambda_w \leftarrow 0$); the actor's adaptive conservatism term $\beta_{\mathrm{eff}} = -2 - \lambda_w \cdot s$ then degenerates to the vanilla RE-SAC value -2 during the warmup window. Rationale: at iteration 0 the ensemble Q-std is tiny (heads have not diverged) and the surprise EMA has not stabilized, so BOCD's belief is nearly uniform and yields a spurious $\lambda_w \approx 0.3$, which under the original $s = 5$ drove β_{eff} to -3.5 (75% more conservative than RE-SAC) from the first gradient step. *P1 — penalty_ scale* 5→2: the default scale in $\beta_{\mathrm{eff}} = -2 - \lambda_w \cdot s$ is reduced from 5 to 2, so after warmup the steady-state $\lambda_w \approx 0.3$ gives $\beta_{\mathrm{eff}} \in [-3.2, -2]$ instead of $[-3.5, -2]$. P0 and P1 are coupled in the sense that fixing only P0 leaves the post-warmup steady-state still over-conservative and fixing only P1 leaves the iteration-0 spike intact; we adopt them together as one logical fix as in the upstream commit.

(B6) **BAPR v14 surprise reform.** Two coupled changes to the BOCD surprise computation. (i) When the training script supplies a `recent_rollout` buffer (the most recent W on-policy transitions, default $W = 256$), the surprise's reward and Q-std signals are computed on that on-policy tail rather than on a random replay batch; under non-stationarity the random batch is dominated by stale-regime transitions and masks the new-regime signal. (ii) The Q-std spike component changes from a two-sided ratio $|q_t/q_{t-1} - 1|$ to a one-sided spike $\max(q_t/q_{t-1} - 1, 0)$. The two-sided form fired during ensemble *convergence* (Q-std dropping), polluting BOCD with false positives; the one-sided form keeps only the genuine regime-change signature.

These six settings form the practical NAU training recipe. The base group (A) is a one-time addition required to make the constrained-head architecture train at all in the BAPR pipeline. Group (B) was developed independently for the BAPR predecessor framework after a multi-seed analysis revealed that $\sim 75\%$ of seeds were getting stuck in poor optimization basins; we found the same diagnosis applies inside the CS-BAPR pipeline, and porting the fixes reduced the residual NAU-specific crash rate (defined as fraction of seeds with m=20× return below −1,500K) from 30% to 10% on the same 10-seed test set (Table 5). In our experiments (Section 6), the full NAU+6-fix configuration (`csbapr`) is what achieves the OOD advantage; an ablation variant (`csbapr-no-nau`) retaining only the same six fixes but with a plain MLP head under-performs at mid-OOD but matches at extreme OOD (Table 2), isolating the architectural contribution.

5.4 (Optional) Pillar II: SINDy symbolic discovery

This subsection describes an optional framework extension that we formalize mathematically but do not activate in our primary experiments. See Section 3 for when to enable it.

The SINDy extension uses Sparse Identification of Nonlinear Dynamics (SINDy) [15] to extract analytic equations from trajectory data, then enforces alignment between the policy's gradient and the discovered law.

Basis library. We define a library of n differentiable candidate functions $\varphi_1, \ldots, \varphi_n : E \to F$ (e.g., $\{1, x, x^2, \sin x, e^x\}$) where each φ_i is differentiable on the domain. The symbolic formula space is the span $\{f_{\text{sym}} = \sum_{i=1}^{n} \xi_i \varphi_i \mid \xi \in \mathbb{R}^n\}$. (Formal tag: `BasisLibrary`.)

SINDy identification. Given trajectory data, SINDy solves:

$$\xi = \arg\min_{\xi} \|\dot{X} - \Theta(X)\xi\|_2 + \lambda\|\xi\|_1, \tag{6}$$

where $\Theta(X)$ is the basis function matrix evaluated on data. The reconstructed formula is $f_{\text{sym}}(x) = \sum_i \xi_i \varphi_i(x)$.

We formalize successful identification as a predicate on output quality:

Definition 5.16 (SINDy Identification Success)**.** *SINDy identifies the true dynamics f_{real} with derivative accuracy ε_1 on domain D if there exist sparse coefficients ξ such that:*

$$\forall x \in D, \quad \|Df_{real}(x) - D(\textstyle\sum_i \xi_i \varphi_i)(x)\| \le \varepsilon_1. \tag{7}$$

(Formal tag: `SINDyIdentifies`.)

Theorem 5.17 (SINDy ⇒ Jacobian Consistency)**.** *If SINDy identifies f_{real} with derivative error ε_1, and the policy π is trained to match the SINDy formula with derivative error ε_2, then π is $(\varepsilon_1 + \varepsilon_2)$-Jacobian Consistent with f_{real}. (Formal tag: `sindy_implies_jacobian_consistency_nD`.)*

Proof sketch. Triangle inequality: $\|D\pi - Df_{\text{real}}\| \le \|D\pi - Df_{\text{sym}}\| + \|Df_{\text{sym}} - Df_{\text{real}}\| \le \varepsilon_2 + \varepsilon_1$. □

Corollary 5.18 (Tight version with same coefficients)**.** *When the policy is trained against the* same *SINDy formula that was identified, i.e., both SINDy error and training error are measured against the same coefficients ξ, the Jacobian Consistency follows directly with the same $\varepsilon_1 + \varepsilon_2$ bound. (Formal tag: `sindy_implies_jacobian_consistency_tight`.) This is the version actually used in the pipeline proof (Theorem 5.19).*

Analytic Consistency Loss. In training, the Jacobian Consistency is enforced through a gradient-alignment loss:

$$\mathcal{L}_{\text{AC}} = \mathbb{E}_{s\sim\mathcal{D}}\left[\|\nabla_s \pi_\theta(s) - \nabla_s f_{\text{sym}}(s)\|^2\right], \tag{8}$$

where f_{sym} is the frozen SINDy formula (its gradients are detached to satisfy the frozen-parameter requirement).

Theorem 5.19 (Full SINDy-to-OOD Pipeline)**.** *If SINDy identifies f_{real} with derivative error ε_1 globally, the policy is trained with derivative error ε_2 globally, $\|\pi(x_0) - f_{real}(x_0)\| \le \delta$, and $\varepsilon_1 + \varepsilon_2 \ge 0$, then:*

$$\|\pi(x_{ood}) - f_{real}(x_{ood})\| \le \delta + (\varepsilon_1 + \varepsilon_2)\|x_{ood} - x_0\|. \tag{9}$$

(Formal tag: `sindy_to_ood_bound_nD`.) This linear bound is the special case of the main quadratic result (Theorem 5.39) when $L = 0$ (pure NAU architecture). When NMU layers are used ($L = 2|c|$), the quadratic term $(L/2)\|d\|^2$ appears.

Proof sketch. By Theorem 5.17, π is $(\varepsilon_1 + \varepsilon_2)$-Jacobian Consistent with f_{real}. Setting $C = \varepsilon_1 + \varepsilon_2$ in Theorem 5.37 yields the result immediately. □

5.5 (Optional) Pillar III: IRM invariance filtering

This subsection describes a sub-extension of the optional SINDy pipeline (Section 5.4). It is active only when the SINDy extension is active and multiple training environments are available. See Section 3.

The IRM extension filters symbolic formula candidates using Invariant Risk Minimization [18] to retain only *cross-environment invariant* physical laws. We use "causal" in the IRM sense of *invariance across environments* [18], not in the structural causal model (SCM) sense of causal discovery [20]. A SINDy formula fitted from a single environment may capture *spurious correlations*—statistical

associations (e.g., temperature $\leftrightarrow$ air density in ballistic control) that hold in the training environment but break under distribution shift. IRM addresses this by requiring the symbolic formula to maintain accuracy across *multiple* environments with distinct dynamics parameters but shared invariant structure.

Definition 5.20 (IRM Consistency)**.** *A SINDy formula with coefficients ξ is ε-*IRM Consistent *across environments $\{e_1, \ldots, e_K\}$ if:*

$$\forall e \in \{e_1, \ldots, e_K\},\ \forall x \in D_e, \quad \|Df^e_{real}(x) - Df_{sym}(x)\| \leq \varepsilon. \tag{10}$$

(Formal tag: `IRM_Consistent`.)

Theorem 5.21 (IRM Tightening)**.** *If formula ξ_{irm} is ε_{irm}-IRM Consistent and formula ξ_{raw} achieves $\varepsilon_{raw} \geq \varepsilon_{irm}$ in a single environment, then the IRM-filtered formula yields Jacobian Consistency bound $\varepsilon_{irm} + \varepsilon_{pol} \leq \varepsilon_{raw} + \varepsilon_{pol}$. (Formal tag: `irm_tightens_jacobian_consistency`.)*

Proof. Direct from $\varepsilon_{\text{irm}} \leq \varepsilon_{\text{raw}}$: $\varepsilon_{\text{irm}} + \varepsilon_{\text{pol}} \leq \varepsilon_{\text{raw}} + \varepsilon_{\text{pol}}$. □

The above theorem is an *if-then* statement; it does not explain *when* IRM achieves $\varepsilon_{\text{irm}} < \varepsilon_{\text{raw}}$. The following result provides a substantive answer via a causal-spurious decomposition:

Theorem 5.22 (IRM Variance Reduction under Causal Decomposition)**.** *Suppose the true dynamics decomposes as $f^e_{real}(x) = f_{causal}(x) + f^e_{spurious}(x)$, where f_{causal} is invariant across environments and $f^e_{spurious}$ varies. Let ε_c be the derivative error of the IRM formula ξ_{irm} against f_{causal} alone. Then:*

(i) ***IRM formula on test env*** *e':* $\|Df^{e'}_{real}(x) - Df^{irm}_{sym}(x)\| \leq \varepsilon_c + \varepsilon^{e'}_s$, *where* $\varepsilon^{e'}_s = \|Df^{e'}_{spurious}(x)\|$ *is the irreducible test-env spurious magnitude.*

(ii) ***Single-env formula on test env*** *e':* $\|Df^{e'}_{real}(x) - Df^{single}_{sym}(x)\| \leq \varepsilon_c + \varepsilon^{e_{train}}_s + \varepsilon^{e'}_s$, *with additional spurious contamination $\varepsilon^{e_{train}}_s$ from the training environment.*

The IRM advantage is $\varepsilon^{e_{train}}_s$: the spurious contamination that IRM filtering removes. This is non-trivial whenever the training environment has non-zero spurious features. (Formal tag: `irm_vs_single_env_gap`, `irm_quantitative_advantage`, *Part XII.)*

Proof sketch. Since $f^{\text{irm}}_{\text{sym}}$ approximates f_{causal} (not $f_{\text{causal}} + f^{e_{\text{train}}}_{\text{spurious}}$), its error on test env e' is $(Df_{\text{causal}} - Df^{\text{irm}}_{\text{sym}}) + Df^{e'}_{\text{spurious}}$, bounded by $\varepsilon_c + \varepsilon^{e'}_s$ via triangle inequality. A single-env formula absorbs $f^{e_{\text{train}}}_{\text{spurious}}$ into its coefficients, introducing an extra $\varepsilon^{e_{\text{train}}}_s$ term when evaluated on a different environment. □

Theorem 5.23 (Full IRM-to-OOD Pipeline)**.** *If formula ξ_{irm} is ε_{irm}-IRM Consistent, the policy is trained with derivative error ε_{pol} against the IRM formula globally, and $\|\pi(x_0) - f_{real}(x_0)\| \leq \delta$ in environment e, then:*

$$\|\pi(x_{ood}) - f^e_{real}(x_{ood})\| \leq \delta + (\varepsilon_{irm} + \varepsilon_{pol})\|x_{ood} - x_0\|. \tag{11}$$

This is the IRM analogue of the SINDy pipeline (Theorem 5.19), composing IRM filtering with the n-D OOD bound. (Formal tag: `irm_to_ood_bound_nD`.*)*

IRM filtering protocol. IRM filtering is applied during Phase 0 (pre-training) and requires $K \geq 3$ environments with distinct dynamics parameters (e.g., different gravity levels, friction coefficients, or traffic demand patterns) but shared causal structure:

1. Collect exploration trajectories from each environment e_k;
2. Fit SINDy independently on each environment's data;
3. Evaluate each candidate formula's derivative error ε_k across *all* environments;

4. Select the coefficient set with the smallest cross-environment derivative variance: $\xi_{\text{irm}} = \arg\min_\xi \text{Var}_k[\varepsilon_k(\xi)]$.

The selected formula is frozen for the entirety of Phase 1 training. We note that this cross-environment variance-minimization selector is a *filtering-time* surrogate for IRM; it does not implement the IRMv1 gradient penalty [18] and is not equivalent to it. A full IRMv1-style selector is a direct drop-in and is formalized in the formal proof at the level of the invariance predicate; we retain the variance surrogate because it is cheap and because our primary experiments do not activate this pipeline (see Section 3).

Remark 5.24 (Environment diversity requirement)**.** *IRM filtering depends critically on environment diversity. With too few environments ($K < 3$) or insufficiently diverse dynamics, IRM cannot distinguish causal from spurious features [18]. In practice, environments should vary in parameters that* do not *affect the target physical law (e.g., varying friction but holding gravity constant when the target law is ballistic). If only a single environment is available, CS-BAPR degrades gracefully: the IRM step is skipped and the single-environment SINDy formula is used directly, with ε_{raw} replacing ε_{irm} in the OOD bound.*

5.6 CS-BAPR Bellman operator

CS-BAPR extends the BAPR operator [1] with a frozen symbolic consistency penalty Γ_{sym}. The mode-conditional operator is:

$$\mathcal{T}_h^{CS} Q(s,a) = R_h(s,a) + \gamma \left(\sum_{s'} P_h(s'|s,a) V^Q(s') - \lambda_{\text{epi}} \Gamma^h_{\text{epi}}(s,a) - \lambda_{\text{sym}} \Gamma^h_{\text{sym}}(s,a) - \kappa_{\text{ale}} \right), \quad (12)$$

where $\Gamma^h_{\text{sym}}(s,a) = \|\nabla_s \pi(s) - \nabla_s f_{\text{sym}}(s)\|^2$ captures the Jacobian deviation between the policy and the SINDy formula. The CS-BAPR operator is the belief-weighted mixture:

$$\mathcal{T}_\rho^{CS\text{-}BAPR} Q(s,a) = \sum_{h \in \mathcal{H}} \rho(h) \cdot \mathcal{T}_h^{CS} Q(s,a). \quad (13)$$

Theorem 5.25 (CS-BAPR Contraction; same template as BAPR)**.** *Under frozen belief $\rho \geq 0$ with $\sum_h \rho(h) = 1$, per-mode stochastic transitions, and frozen penalties (κ_{ale}, Γ_{epi}, Γ_{sym}), the operator $\mathcal{T}_\rho^{CS\text{-}BAPR}$ is a γ-contraction in L_∞ with a unique fixed point. (Formal tag: `csbapr_contraction`.)*

Proof sketch. All three penalty terms (Γ_{epi}, Γ_{sym}, κ_{ale}) are frozen during each backup and therefore cancel in the contraction argument:

$$\begin{aligned} &|\mathcal{T}_h^{CS} Q_1(s,a) - \mathcal{T}_h^{CS} Q_2(s,a)| \\ &\quad = \gamma \left| \sum_{s'} P_h(s'|s,a)(V^{Q_1}(s') - V^{Q_2}(s')) \right| \leq \gamma \varepsilon. \end{aligned} \quad (14)$$

The mixture over modes preserves contraction since ρ is a probability distribution. The full proof, including Blackwell's sufficiency conditions, is formal. □

Remark 5.26 (Marginal extension over BAPR's contraction)**.** *Theorem 5.25 adds one frozen penalty Γ_{sym} to BAPR's already-frozen penalty stack ($\Gamma_{epi}, \kappa_{ale}$); the proof template (frozen-penalty cancellation in $\mathcal{T}(Q_1) - \mathcal{T}(Q_2)$) is identical to BAPR's and to RE-SAC's before that, as we make explicit in Theorem 5.29. The contribution of this theorem is bookkeeping—the new Γ_{sym} does not break the existing γ-contraction property, which our formalization verifies—rather than a new mathematical idea. We retain the theorem statement for completeness because the algorithm in Section 5.9 relies on it, but readers familiar with the BAPR contraction can take the result on inspection.*

Corollary 5.27 (Contraction after belief update)**.** *If the belief ρ is updated via Bayesian normalization $\rho'(h) = \rho(h) \cdot L(h)/Z$ with non-negative likelihoods and positive normalizer, then $\mathcal{T}_{\rho'}^{CS\text{-}BAPR}$ remains a γ-contraction. (Formal tag: `csbapr_contraction_after_update`.)*

Theorem 5.28 (Necessity of frozen Γ_{sym}: contraction breaks under Q-dependent symbolic penalty)**.** *Consider the CS-BAPR operator on a single-state single-action MDP where Γ_{sym} depends on Q with coupling sensitivity $\mu \geq 0$, defined as $\mu = |\partial\Gamma_{sym}/\partial Q|$ evaluated at the relevant stationary point (i.e., $\Gamma_{sym}(Q) \approx C - \mu Q$ due to online SINDy refinement creating a feedback loop: higher $Q \to$ better policy $\to$ lower Jacobian error $\to$ lower Γ_{sym}). Then the effective contraction factor is $\gamma(1 + \lambda_{sym}\mu)$, and:*

(i) If $\lambda_{sym}\mu \geq 1/\gamma - 1$: the operator is not *a contraction;*

(ii) If $\lambda_{sym}\mu < 1/\gamma - 1$: the operator is *a contraction with factor $\gamma(1 + \lambda_{sym}\mu) > \gamma$.*

The boundary $\gamma(1 + \lambda_{sym}\mu) = 1$ is sharp; Theorem 5.29 gives the shared algebraic template.

Proof sketch. When $\Gamma_{\text{sym}}(Q) = C - \mu Q$, the penalty term $-\lambda_{\text{sym}}\Gamma_{\text{sym}}(Q)$ contributes $+\lambda_{\text{sym}}\mu Q$ to the operator. The simplified operator becomes $\mathcal{T}_{\text{bad}}(Q) = \gamma(1 + \lambda_{\text{sym}}\mu) \cdot Q + R_0$ for constant R_0. Taking witnesses $Q_1 = 1, Q_2 = 0$: $|\mathcal{T}_{\text{bad}}(1) - \mathcal{T}_{\text{bad}}(0)| = \gamma(1 + \lambda_{\text{sym}}\mu) \geq 1$. □

Remark 5.29 (Structural unity of frozen-parameter counterexamples)**.** *All three counterexamples in this paper series share the same algebraic structure:*

Paper	***Non-frozen term***	***Factor***	***Counterexample tag***
RE-SAC	$\kappa(Q) = \lambda Q$	$\gamma + \lambda$	`resac_nonfrozen`
BAPR	$\rho(Q)$ + *reward gap* Δ	$\gamma + \lambda\Delta$	`bapr_nonfrozen`
CS-BAPR	$\Gamma_{sym}(Q)$ *feedback*	$\gamma(1 + \lambda_{sym}\mu)$	`csbapr_nonfrozen`

In every case, the frozen penalty cancels exactly in $\mathcal{T}(Q_1) - \mathcal{T}(Q_2)$ (yielding factor γ), but a Q-dependent version introduces a coupling term that pushes the factor above γ—and past 1 when the coupling is strong enough. The frozen-parameter design is therefore necessary *for contraction in all three frameworks.*

Remark 5.30 (Tractable approximation: shared Γ_{sym} across modes)**.** *The formal proof indexes Γ_{sym} by mode h (i.e., $\Gamma^h_{sym}(s, a)$), allowing mode-specific symbolic penalties. In the implementation, we use a* single shared Γ_{sym} *across all modes, since the SINDy formula f_{sym} is designed to be causally invariant (via IRM filtering) and therefore mode-independent. This is a conservative approximation: the formal proof holds for* arbitrary *values of Γ^h_{sym}; using identical values across modes is a strict special case. The approximation affects the quality of the fixed point (how well Γ_{sym} captures mode-specific Jacobian deviations) but not the existence or uniqueness of the fixed point.*

5.7 Component composition: from pillars to bound hypotheses

Before stating the main OOD theorem, we show how the three pillars combine to satisfy its hypothesis—specifically, the linear derivative error growth condition.

Proposition 5.31 (Component hypotheses imply derivative bound (generalized))**.** *Under Assumptions 5.3–5.5, if:*

(i) f_{real} has M-Lipschitz derivatives: $\|Df_{real}(x_1) - Df_{real}(x_2)\| \leq M\|x_1 - x_2\|$ (Assumption 5.4),

(ii) The policy has ε-Jacobian Consistency at the training boundary x_0 (from SINDy + training, Theorem 5.17),

(iii) The policy has L_{pol}-Arithmetic Inductive Bias (from NAU/NMU architecture, Assumption 5.5),

then for all *x (including OOD points never seen during training):*

$$\|D\pi(x) - Df_{real}(x)\| \leq \varepsilon + (L_{pol} + M) \cdot \|x - x_0\|. \tag{15}$$

When $M = 0$ (linear physics) and $L_{pol} = 0$ (pure NAU), this gives the constant bound ε. (Formal tag: `deriv_error_propagation`, `deriv_error_propagation_nD`, Part X.)

Proof sketch. Triangle inequality on three terms: $\|D\pi(x) - Df_{\text{real}}(x)\| \leq \|D\pi(x) - D\pi(x_0)\| + \|D\pi(x_0) - Df_{\text{real}}(x_0)\| + \|Df_{\text{real}}(x_0) - Df_{\text{real}}(x)\| \leq L_{\text{pol}}\|x - x_0\| + \varepsilon + M\|x - x_0\|$. □

Proposition 5.32 (n-D composition: from pillars to OOD bound (no OOD assumptions))**.** *Under Assumptions 5.3–5.5, if:*

(i) π *has* L_{pol}*-Arithmetic Inductive Bias in the n-D sense (Definition 5.8),*

(ii) π *is* ε*-Jacobian Consistent with* f_{real} at the training boundary x_0 *(from training loss* $\mathcal{L}_{AC}$*),*

(iii) f_{real} *has* M*-Lipschitz Fréchet derivatives (Assumption 5.4),*

then by Proposition 5.31, the global derivative error is $\|D\pi(x) - Df_{real}(x)\| \leq \varepsilon + (L_{pol} + M)\|x - x_0\|$ *for all* x*. Setting* $C = \varepsilon + (L_{pol} + M)\|d\|$ *as a global bound on the segment from* x_0 *to* x_{ood}*, the n-D linear OOD bound (Theorem 5.37) yields:*

$$\|\pi(x_{ood}) - f_{real}(x_{ood})\| \leq \delta + \big(\varepsilon + (L_{pol} + M)\|d\|\big) \cdot \|d\|. \tag{16}$$

(Formal tag: `ood_bound_no_assumption4_nD`*, Part X.)*

Remark 5.33 (Relation to Theorem 5.36)**.** *Expanding the bound in Proposition 5.31 yields* $\delta + \varepsilon\|d\| + (L_{pol} + M)\|d\|^2$*, where the quadratic coefficient is* $(L_{pol} + M)$*. In contrast, the fencing-based Theorem 5.36 obtains* $\delta + \varepsilon\|d\| + \frac{L}{2}\|d\|^2$ *with* $L = L_{pol} + M$*, giving a tighter factor of* $\frac{1}{2}$ *on the quadratic term. The difference arises because the Proposition applies the MVT with a* worst-case constant $C = \varepsilon + (L_{pol} + M)\|d\|$ *over the entire segment, while the Theorem exploits the* linearly growing *derivative error* $\varepsilon + L\|x - x_0\|$ *via integration, recovering the* $\frac{1}{2}$ *factor. In experiments (Section 6), we report both bounds; the Proposition bound serves as a conservative envelope and the Theorem bound as a tighter prediction.*

This n-D composition proposition provides the formal bridge between the three pillars (which supply ε, L_{eff}, and the $M = 0$ idealization) and the path-level hypothesis consumed by the fencing theorems. The causal chain is: **NAU/NMU** (Pillar I) provides finite L_{eff}; **SINDy** (Pillar II) provides small $\varepsilon = \varepsilon_1 + \varepsilon_2$; **IRM** (Pillar III) tightens ε by replacing $\varepsilon_1^{\text{raw}}$ with $\varepsilon_1^{\text{irm}} \leq \varepsilon_1^{\text{raw}}$; and **RL training** provides small δ. All four quantities enter the final bound.

Remark 5.34 (On the nature of the theoretical contribution)**.** *The algebraic core of the OOD bound is an application of the Mean Value Theorem and Grönwall-type fencing. The theoretical contribution lies not in the calculus but in the* structural design insight*: identifying that the specific combination of NAU/NMU architecture, SINDy alignment, and IRM filtering makes the MVT hypotheses* satisfiable*—and proving that the standard ReLU architecture makes them* provably unsatisfiable*. The formal verification serves as a design audit tool, forcing every structural assumption to be explicit, much as it did for the frozen-parameter requirement in BAPR [1].*

5.8 Main theorem: OOD extrapolation stability

We now state the main theoretical result, beginning with the key building blocks. The mathematical content of this section is a careful instantiation of two textbook calculus tools—the convex Mean Value Theorem and a Grönwall-style fencing argument for F-valued curves—specialized to the policy-vs-reference error function with the explicit $(\delta, \varepsilon, L, d)$ dependence required for the architectural argument in Section 5.2. We do not claim the calculus underlying the bound is itself novel; the contribution is in identifying the dependence structure that lets a $\{\text{NAU}, \text{NMU}, \text{KAN}, \text{tanh-MLP}, \text{ReLU}\}$-typed comparison fall out of one inequality.

Lemma 5.35 (Linear error growth via MVT)**.** *Let* $\pi, f_{real} : \mathbb{R} \to \mathbb{R}$ *be differentiable, and suppose* $\|\pi'(x) - f'_{real}(x)\| \leq C$ *for all* $x \in [a, b)$*. Then* $\|(\pi(b) - f_{real}(b)) - (\pi(a) - f_{real}(a))\| \leq C \cdot (b - a)$.

This fundamental lemma establishes that bounded derivative error implies bounded error growth. Combined with the base accuracy δ, it gives the linear OOD bound. The quadratic generalization below replaces the constant C with a *growing* derivative error $\varepsilon + L(x - a)$.

Theorem 5.36 (1D OOD Extrapolation Stability Bound)**.** *Let $\pi, f_{real} : \mathbb{R} \to \mathbb{R}$ be differentiable. If:*

(i) $\|\pi(a) - f_{real}(a)\| \le \delta$ *(base accuracy),*

(ii) $\forall x \in [a, b),\ \|\pi'(x) - f'_{real}(x)\| \le \varepsilon + L(x - a)$ *(growing derivative error),*

then: $\|\pi(b) - f_{real}(b)\| \le \delta + \varepsilon(b - a) + \frac{L}{2}(b - a)^2$.
This is the one-dimensional fencing bound used by the n-D theorem below.

Proof sketch. Define the error function $e(x) = \pi(x) - f_{\text{real}}(x)$. Since $\|e(a)\| \le \delta$ and $\|e'(x)\| \le \varepsilon + L(x - a)$ for $x \in [a, b)$, apply the Grönwall-style fencing lemma with envelope $g(x) = \delta + \varepsilon(x - a) + \frac{L}{2}(x - a)^2$. Verify: $g(a) = \delta \ge \|e(a)\|$ and $g'(x) = \varepsilon + L(x - a) \ge \|e'(x)\|$. The conclusion $\|e(b)\| \le g(b)$ gives the bound. □

Theorem 5.37 (n-D Linear OOD Bound)**.** *For $\pi, f_{real} : E \to F$ in normed spaces, if $\|\pi(x_0) - f_{real}(x_0)\| \le \delta$ and $\forall z,\ \|D\pi(z) - Df_{real}(z)\| \le C$, then:*

$$\|\pi(x_{ood}) - f_{real}(x_{ood})\| \le \delta + C\|x_{ood} - x_0\|. \tag{17}$$

(Formal tag: `scbapr_ood_linear_nD`. Proof uses Mathlib's convex Mean Value Theorem.)

Proof sketch. Define $e(x) = \pi(x) - f_{\text{real}}(x)$, so $De(x) = D\pi(x) - Df_{\text{real}}(x)$ with $\|De(x)\| \le C$. Apply Mathlib's convex MVT (`Convex.norm_image_sub_le_of_norm_fderiv_le`): $\|e(x_{\text{ood}}) - e(x_0)\| \le C\|x_{\text{ood}} - x_0\|$. Triangle inequality: $\|e(x_{\text{ood}})\| \le \|e(x_0)\| + C\|x_{\text{ood}} - x_0\| \le \delta + C\|d\|$. □

Lemma 5.38 (Path-level fencing theorem)**.** *Let $e_{path} : \mathbb{R} \to F$ be the error function along a line segment $[0, 1]$ (typically $e_{path}(t) = \pi(x_0 + t \cdot d) - f_{real}(x_0 + t \cdot d)$). If $\|e_{path}(0)\| \le \delta$ and $\|e'_{path}(t)\| \le \varepsilon_d + L_d \cdot t$ for $t \in [0, 1)$, then: $\|e_{path}(1)\| \le \delta + \varepsilon_d + L_d/2$. This is the F-valued generalization of the 1D bound (Theorem 5.36), applied to the composed path. (Formal tag: `scbapr_ood_quadratic_path`.)*

The proof chain is: **1D fencing** (Theorem 5.36) → **path-level fencing** (Lemma 5.38) → **n-D bound** (Theorem 5.39 below), where the final step substitutes $\varepsilon_d = \varepsilon\|d\|$ and $L_d = L\|d\|^2$.

Theorem 5.39 (n-D Quadratic OOD Bound (Main Result))**.** *For $\pi, f_{real} : E \to F$, if along the line segment from x_0 to x_{ood}:*

(i) $\|\pi(x_0) - f_{real}(x_0)\| \le \delta$,

(ii) The derivative error grows linearly: $\|e'(t)\| \le \varepsilon\|d\| + L\|d\|^2 \cdot t$ *for* $t \in [0, 1)$,

where $d = x_{ood} - x_0$, then:

$$\|\pi(x_{ood}) - f_{real}(x_{ood})\| \le \delta + \varepsilon\|d\| + \frac{L}{2}\|d\|^2. \tag{18}$$

(Formal tag: `scbapr_ood_quadratic_nD`.)

Proof sketch. Parametrize the line segment as $\gamma(t) = x_0 + t \cdot d$ for $t \in [0, 1]$ and define $e_{\text{path}}(t) = \pi(\gamma(t)) - f_{\text{real}}(\gamma(t))$. Then $\|e_{\text{path}}(0)\| \le \delta$ and $\|e'_{\text{path}}(t)\| \le \varepsilon\|d\| + L\|d\|^2 t$ by hypothesis (ii). Apply the path-level fencing lemma (Lemma 5.38) with $\varepsilon_d = \varepsilon\|d\|$ and $L_d = L\|d\|^2$: $\|e_{\text{path}}(1)\| \le \delta + \varepsilon_d + L_d/2 = \delta + \varepsilon\|d\| + \frac{L}{2}\|d\|^2$. Since $\gamma(1) = x_{\text{ood}}$, this gives $\|\pi(x_{\text{ood}}) - f_{\text{real}}(x_{\text{ood}})\| \le \delta + \varepsilon\|d\| + \frac{L}{2}\|d\|^2$. Full proof in Appendix B. □

Remark 5.40 (Engineering interpretation)**.** *The bound (18) decomposes OOD error into three controllable terms, each addressed by a specific CS-BAPR component:*

Term	***Meaning***	***Controlled by***
δ	*Training-domain accuracy*	*Standard RL training*
$\varepsilon\|d\|$	*SINDy quality × distance*	*Pillar II (SINDy) + III (IRM)*
$\frac{L}{2}\|d\|^2$	*Architectural extrapolation*	*Pillar I (NAU: L=0, NMU: L=2|c|)*

With NAU ($L = 0$), the bound is linear *in distance. With NMU, it is* quadratic. *With ReLU, no finite L exists within this framework (Theorem 5.13), so the bound cannot be instantiated. This hierarchy provides a principled criterion for architecture selection: the trade-off is expressiveness (NMU can represent quadratic functions that NAU cannot) vs. extrapolation tightness ($L = 0$ for NAU vs. $L = 2|c|$ for NMU).*

Remark 5.41 (Theoretical role vs. empirical contribution of each pillar)**.** *Each pillar plays a distinct* theoretical *role in the bound, but their empirical contributions in our experiments differ markedly:*

- ***Pillar I (NAU/NMU output head):*** *controls L in the quadratic term. ReLU gives $L = \infty$ (Theorem 5.13), so the bound becomes vacuous; smooth activations (GELU, Softplus, tanh) admit finite L but pay a non-zero quadratic-term penalty that NAU avoids by construction.* Empirically: *Pillar I helps in mid-range OOD on the bus benchmark and on tasks whose optimal policy is approximately scale-linear-in-state, but is neutral or harmful on tasks whose optimal policy must structurally re-tune (Section 6.5).*

- ***Pillar II (SINDy-based Jacobian Consistency):*** theoretically *tightens the linear-term coefficient ε by replacing the policy's own linear extrapolation with an identified symbolic reference.* Empirically: *the action-aware SINDy configuration (`sindy_with_control = True`, fitting $\dot{x} = Ax + Bu$ jointly) on the LQR benchmark gives the symbolic alignment a 66−97% mid-OOD improvement over the no-SINDy ablation (Table 6)—the first positive empirical case we have found. The earlier state-only configuration (default $n_{control} = 0$) was a negative result on the same benchmark, because the symbolic reference encoded the open-loop A rather than the closed-loop $A - BK$ that the optimal policy implements; the action-aware fit moves the reference into the regime in which K is representable. The full closed-loop alignment ($\nabla\pi \approx -K$ via Riccati-solving K post-Phase-0) is a tighter formulation flagged for the next paper revision. The transit benchmark itself is outside the SINDy-applicable regime (counteraction task; Section 3).*

- ***Pillar III (IRM filter):*** theoretically *tightens ε further via cross-environment filtering; not required for the bound to be finite.* Empirically: *not activated in our primary benchmarks (transit holding has a single regime); demonstration deferred to a multi-regime variant.*

The bound itself is structurally vacuous *only when Pillar I is removed (since $L \to \infty$ for ReLU). The other two pillars affect ε, which is controlled empirically by training and remains finite without symbolic alignment. The graceful degradation hierarchy (Theorem 5.43) ensures partial configurations remain usable.*

5.9 CS-BAPR algorithm

The complete CS-BAPR algorithm operates in two phases:

Phase 0: SINDy Pre-Identification. Before RL training begins, the agent collects exploration trajectories using a random policy, fits a SINDy model to discover sparse dynamics, and optionally applies IRM filtering across multiple environments. The resulting symbolic formula f_{sym} is frozen as a PyTorch module with detached gradients.

Phase 1: CS-BAPR Training. The training loop extends BAPR with two additions: (1) the symbolic consistency penalty Γ_{sym} in the Q-loss (Eq. (12)), and (2) the Jacobian Consistency Loss $\mathcal{L}_{\text{AC}}$ (Eq. (8)) in the policy loss.

Algorithm 1 CS-BAPR: Causal-Symbolic BAPR

Phase 0: SINDy Pre-Identification
Collect exploration trajectories $\{(s_t, a_t, s_{t+1})\}$ using random policy.
Fit SINDy: $\xi = \arg\min \|\dot{X} - \Theta(X)\xi\|_2 + \lambda\|\xi\|_1$.
(Optional) IRM filter: select ξ with minimum cross-environment variance.
Create frozen PyTorch wrapper: $f_{\text{sym}} \leftarrow$ `SINDyTorchWrapper`(ξ).

Phase 1: CS-BAPR Training Loop
Initialize: NAU/NMU Actor π_θ, Ensemble critics $\{Q_{\phi_k}\}$, BOCD belief ρ.
for each training iteration **do**
  Collect transitions; store in $\mathcal{D}$.
  Compute surprise ξ_t; update BOCD belief ρ (frozen during backup).
  Compute λ_w and β_{eff} from belief (inherited from BAPR).
  for $n = 1, \ldots, N_{\text{updates}}$ **do**
    Sample mini-batch from $\mathcal{D}$.
    // Critic update (with Γ_{sym}):
    $\Gamma_{\text{sym}} \leftarrow \|\nabla_s \pi_\theta(s') - \nabla_s f_{\text{sym}}(s')\|^2$ (SINDy gradients detached).
    $y \leftarrow r + \gamma(\bar{Q}_{\text{target}} - \alpha \log \pi - \lambda_{\text{epi}}\Gamma_{\text{epi}} - \lambda_{\text{sym}}\Gamma_{\text{sym}} - \kappa)$.
    Update ϕ_k by minimizing $\mathcal{L}_Q + \beta_{\text{ood}}\sigma_{\text{ens}}$.
    // Actor update (with $\mathcal{L}_{AC}$):
    $\mathcal{L}_\pi \leftarrow -(\bar{Q} + \beta_{\text{eff}}\sigma_{\text{ens}} - \alpha \log \pi) + \mu_{\text{jac}} \cdot \mathcal{L}_{\text{AC}}$.
    Update θ by minimizing $\mathcal{L}_\pi$ (with gradient clipping).
    Soft update target networks.
  end for
end for

Remark 5.42 (Frozen-parameter requirements)**.** *CS-BAPR requires four frozen-parameter conditions for contraction:*

(i) κ_{ale} *frozen (from RE-SAC target weights);*

(ii) Γ_{epi} *frozen (from target Q-ensemble);*

(iii) ρ *frozen during backup (from BOCD, updated between iterations);*

(iv) Γ_{sym} *frozen (SINDy coefficients fixed throughout training).* ***[CS-BAPR new]***

Violating any one of these conditions breaks the contraction guarantee, as established by the formal counterexamples in BAPR [1], RE-SAC [2], and CS-BAPR (Theorem 5.28).

Remark 5.43 (Graceful degradation)**.** *CS-BAPR is designed to degrade gracefully when its assumptions are not met:*

- ***SINDy fails*** *($\varepsilon_1 > \varepsilon_{threshold}$): Γ_{sym} and $\mathcal{L}_{AC}$ are disabled; the algorithm reduces to BAPR with a standard MLP or NAU/NMU actor, retaining all BAPR convergence guarantees.*
- ***No multiple environments****: IRM filtering is skipped; single-environment SINDy formula used directly, with ε_{raw} replacing ε_{irm} in the bound.*
- ***NAU/NMU disabled****: The architecture reverts to ReLU; the OOD bound no longer holds, but the Bellman contraction guarantee is unaffected (it depends only on frozen penalties, not on the actor architecture).*

This hierarchical fallback ensures that CS-BAPR is never worse *than BAPR, even when its additional components cannot be applied.*

Remark 5.44 (Computational overhead)**.** *CS-BAPR adds three sources of overhead relative to BAPR:*

- ***Phase 0 (SINDy, one-time):*** *50 exploration episodes + PySINDy fitting, typically <5 minutes. Amortized over the full training, this is negligible.*
- $\mathcal{L}_{AC}$ ***(per gradient step):*** *One additional* `autograd` *call to compute* $\nabla_s \pi_\theta(s)$*. For the action-dimension output (*$\dim(\mathcal{A}) \ll \dim(\mathcal{S})$*), this is* $O(\dim(\mathcal{S}))$ *per sample, comparable to a single backward pass.*
- ***NAU regularization (per step):*** $O(1)$*—a scalar penalty on weight magnitudes.*

In wall-clock time, CS-BAPR is approximately 15–20% slower than BAPR (the dominant cost remains the ensemble critic's K forward/backward passes). The Jacobian computation can be further accelerated via JVP with random vectors for high-dimensional state spaces, reducing cost from $O(\dim(\mathcal{S})^2)$ *to* $O(\dim(\mathcal{S}))$*.*

6 Experiments

We evaluate the primary NAU+stabilization pipeline on a transit holding control benchmark with magnitude-type OOD. The optional symbolic extensions (SINDy and IRM) are *not* activated in these experiments, for the reason discussed in Section 3: transit holding is a counteraction task, where the optimal policy opposes the identified dynamics rather than tracking them, and an earlier internal variant that activated Γ_{sym} under-performed the NAU-only configuration. We therefore position this section as a validation of the architectural bound, not of the full framework.

Four questions: **(Q1)** Does the NAU actor beat the ReLU BAPR baseline under in-distribution operation? **(Q2)** Does it beat BAPR under magnitude OOD (demand multipliers 5−50×)? **(Q3)** Is the NAU head necessary, or do the training-stabilization fixes alone explain the advantage? **(Q4)** Is the empirical OOD degradation consistent with the bound $\delta + \varepsilon\|d\| + \frac{L}{2}\|d\|^2$?

6.1 Benchmark: SUMO-calibrated 12-line transit holding

Environment. We use `MultiLineEnv`, a multi-line transit holding simulator built on SUMO-calibrated headway/dwell dynamics for 12 real urban bus lines (variable route length, station spacing, and schedule). At each station-arrival event, the agent chooses a hold duration for the arriving bus; the environment advances to the next event across the fleet. Passenger arrivals follow a Poisson process with line-specific rates calibrated to operational data. The state is a 15-D continuous vector (preceding/following headway, load, expected boarding rate, upcoming segment speed, cumulative delay, previous action, and categorical descriptors for line / station / time); the action is a 1-D hold-time scalar in $[0, h_{\max}]$ (we set $h_{\max} = 60\,\text{s}$).

Why bus is approximately scale-linear in state. The transit holding task has a structural property that makes it well-matched to NAU's $L = 0$ output head: the strong holding policy maps state to hold time *approximately linearly*. The classical analytic holding controller [24, 25] takes the form $h^* = \alpha(h_{\text{prev}} - h^{\text{target}})^+ + \beta(\text{load deficit})^+$, a piecewise-linear function of state. Empirically, we measured the linear-fit quality on a strong learned controller (the best v14 `csbapr` checkpoint, run deterministically over five ID episodes for 47,247 decisions): on a 50/50 train/test split, ordinary least squares regression of hold time on the 15-D normalized state vector achieves $R^2 = 0.70$ on the held-out half, with 0% of decisions hitting either the 0 s or 60 s boundary clamps under in-distribution flow. Since the demand multiplier m scales the Poisson rate, which in turn scales boarding-induced dwell, headway deviation, and (in the analytic form above) the optimal hold all roughly linearly with m, magnitude OOD on this benchmark is the regime in which a constant-derivative output head can extrapolate at zero second-order cost. Saturation at $h_{\max} = 60\,\text{s}$ caps the regime in which this argument applies (we observe boundary clamping only at $m \geq 20\times$), and the diminishing returns of NAU at m=50× in Table 2 are consistent with this saturation. This is the structural reason NAU helps in the bus benchmark and not in Hopper-wind (Section 6.5), where the optimal policy must re-tune the gait rather than scale a state-linear response, nor in LQR after action saturation kicks in (Section 6.5).

Table 1: In-distribution return (per-line eval at $m = 1$). Higher is better. Mean ± s.d., n=10 seeds per row.

Method	n seeds	ID total return (K)
`bapr-pristine` (ReLU + no fixes)	10	-850 ± 37
`bapr` (legacy, Group-A only)	10	-793 ± 34
`csbapr-no-nau` (ReLU-MLP + 6 fixes)	10	-780 ± 55
`csbapr` (NAU + 6 fixes)	10	-772 ± 36
`csbapr-tanh` (smooth-MLP + 6 fixes)	10	-817 ± 54
`csbapr-kan` (KAN + 6 fixes)	10	-759 ± 35

OOD protocol. We control OOD magnitude via a single demand multiplier m applied to the Poisson arrival rate in the environment's `_batch_passenger_arrival` routine, with two variants:

- **Static sweep:** the multiplier $m \in \{1, 2, 5, 10, 20, 50\}$ is held constant for the full 18,000-second episode; $m = 1$ is the in-distribution setting.
- **Abrupt burst:** $m = 1$ for $t \in [0, 3600)$ s, then $m \in \{5, 10, 20, 50\}$ for $t \geq 3600$ s. This models a sudden surge (mega-event, service disruption) partway through operation.

Methods. We compare six training configurations, all sharing the BAPR training loop (BOCD belief tracker, ensemble critic, frozen-penalty contraction) and differing in actor head and stabilization toggles:

(i) `bapr-pristine`: ReLU-MLP actor with all six stabilization toggles disabled.

(ii) `bapr`: legacy BAPR preset, retained for back-compat because some Group-A fixes were already active in the previous code state.

(iii) `csbapr-no-nau`: ReLU-MLP actor + all six stabilization fixes; this isolates the stabilization contribution.

(iv) `csbapr` **(primary)**: NAU actor + all six stabilization fixes.

(v) `csbapr-tanh`: smooth MLP-with-tanh + all six stabilization fixes.

(vi) `csbapr-kan`: KAN actor (learnable B-spline edges) + all six stabilization fixes.

The optional symbolic extensions (Γ_{sym}, $\mathcal{L}_{\text{AC}}$, IRM) are disabled in all six configurations: `jac_weight` = 0, `weight_sym` = 0. For all six configurations we run n=10 seeds for 500 training episodes per seed on the 12-line benchmark, and evaluate the best checkpoint on the OOD sweep, abrupt-burst, and oscillating protocols described above. Unless stated otherwise, the OOD tables report medians over seeds; Table 1 reports mean ± s.d.

6.2 In-distribution convergence (Q1)

Table 1 reports best-of-training return at $m = 1$ across the six configurations. The NAU actor converges to comparable in-distribution return as the stabilized MLP variants, while KAN gives the strongest in-distribution score. The stabilization fixes alone (`csbapr-no-nau`) substantially improve over the clean `bapr-pristine` baseline, as expected.

6.3 Magnitude OOD (Q2, Q3)

Table 2 reports return under the static demand sweep; Table 3 under the abrupt burst; Table 4 under within-episode oscillation; Table 5 the crash rate at m=20×. With n=10 seeds across all six methods we now have a clean ablation. We observe:

Table 2: Static OOD demand sweep, total episode return (K, higher is better). Median over n=10 seeds per row. $m = 1$ is in-distribution. `bapr-pristine` disables all six stabilization toggles. `bapr` is the legacy preset whose Group-A fixes were silently always-on under the prior code state, retained for back-compat. **Bold**: best in column.

Method	m=1	m=2	m=5	m=10	m=20	m=50
`bapr-pristine` (ReLU, no fixes)	−861	−862	−939	−1134	−1267	−1416
`bapr` (legacy, Group-A only)	−791	−799	−878	−1025	−1196	−1428
`csbapr-no-nau` (ReLU-MLP+6 fixes)	−770	−776	−827	**−919**	**−1076**	−1314
`csbapr` (NAU+6 fixes)	−772	−778	**−807**	−950	−1098	−1293
`csbapr-tanh` (smooth-MLP+6 fixes)	−786	−824	−897	−986	−1087	**−1239**
`csbapr-kan` (KAN+6 fixes)	**−756**	**−762**	−820	−971	−1121	−1335

Table 3: Abrupt-burst protocol: $m = 1$ for $t \in [0, 3600)$ s, then abrupt switch to the specified multiplier. Total episode return (K), median over n=10 seeds.

Method	burst 5×	burst 10×	burst 20×	burst 50×
`bapr-pristine`	−909	−1021	−1178	−1354
`bapr` (legacy)	−836	−963	−1104	−1316
`csbapr-no-nau`	−792	**−858**	**−982**	−1205
`csbapr` (NAU)	−792	−881	−1000	−1196
`csbapr-tanh`	−847	−932	−1017	**−1152**
`csbapr-kan`	**−790**	−909	−1054	−1224

(i) *Six-fix recipe alone is the dominant empirical contribution.* Holding the architecture fixed at ReLU-MLP, the six stabilization toggles take the median ID return from −861K (`bapr-pristine`) to −770K (`csbapr-no-nau`)—a 10.6% improvement at m=1 that opens to 11.9% at m=5, 19.0% at m=10, 15.1% at m=20, and 7.2% at m=50. The legacy `bapr` preset (Group-A subset of fixes silently always-on under prior code) sits in between, validating that the gap is a continuum across how many fixes are activated.

(ii) *No single architecture dominates across the OOD axis.* `csbapr-kan` is best at m=1−2; `csbapr` (NAU) is best at m=5; `csbapr-no-nau` (ReLU-MLP) is best at m=10−20; `csbapr-tanh` (smooth-MLP) is best at m=50. We read this as a Pareto trade-off rather than a winner: NAU's L=0 helps in the mid-OOD regime where action saturation has not yet kicked in, smooth-MLP's bounded L is most useful at extreme OOD where the policy needs gentle non-linearity to absorb saturation, and KAN's spline expressivity gives a small ID/near-OOD edge at the cost of higher gradient variance at $m \geq 10$.

(iii) Quantitatively, `csbapr` (NAU + 6 fixes) improves over `bapr-pristine` by 14.1% at m=1, 14.0% at m=5, 16.2% at m=10, 13.3% at m=20, and 8.7% at m=50. The 6-fix component (`csbapr-no-nau` vs. `bapr-pristine`) accounts for 10.6% at m=1, 11.9% at m=5, 19.0% at m=10, 15.1% at m=20, and 7.2% at m=50; the architectural addition (`csbapr` vs. `csbapr-no-nau`) is small in magnitude and Pareto-trades against `csbapr-tanh` and `csbapr-kan` as discussed in (ii).

(iv) `csbapr-kan`'s −756K ID is the best in-distribution number we measure, but the spline derivatives' non-zero L produces visible $L\|d\|^2/2$ widening at $m \geq 10$ (−971K vs. `csbapr-no-nau`'s −919K), and one of ten seeds crashes at −1,551K under m=20 (Table 5). NAU's L=0 avoids both.

The crash table shows two distinct effects. The tighter −1,200K threshold mostly tracks per-method performance level, where the stabilized methods reduce the fraction of poor seeds relative to `bapr-pristine`. The deeper thresholds expose rare outliers: `csbapr` has one severe NAU seed at −1,851K, `csbapr-kan` has one seed below −1,500K, and no method produces a seed below −2,000K.

Reading the tables. All four CS-BAPR variants (`-no-nau`, , `-tanh`, `-kan`) improve over the clean `bapr-pristine` baseline across the primary OOD protocols, but the best variant changes with the

Table 4: Oscillating OOD protocol: within-episode piecewise-constant demand schedule. Total episode return (K), median over n=10 seeds. Schedules: `commuter_day` has 5 switches with peak 50×; `square_wave_20x` has 9 switches with peak 20×; `escalating_10x` has 4 switches with peak 50×.

Method	`commuter_day`	`square_wave_20x`	`escalating_10x`
`bapr-pristine`	−998	−1111	−956
`bapr` (legacy)	−954	−1045	−920
`csbapr-no-nau`	**−859**	**−921**	**−845**
`csbapr` (NAU)	−865	−927	−853
`csbapr-tanh`	−929	−990	−908
`csbapr-kan`	−914	−972	−912

Table 5: Crash rate at m=20× across multiple severity thresholds. Each cell reports the fraction of seeds whose return at m=20× falls below the threshold (lower is better). All six methods use n=10 seeds. We chose threshold values spanning the BAPR-baseline median so the qualitative ranking is robust to threshold choice; thresholds tighter than −1,200K are dominated by the level of the per-method ID return.

Method	n	$< -1{,}200$K	$< -1{,}500$K	$< -1{,}800$K	$< -2{,}000$K	Worst seed (K)
`bapr-pristine` (ReLU, no fixes)	10	70%	0%	0%	0%	−1,422
`bapr` (legacy, Group-A only)	10	50%	0%	0%	0%	−1,367
`csbapr-no-nau` (ReLU-MLP+6 fixes)	10	40%	0%	0%	0%	−1,309
`csbapr` (NAU+6 fixes)	10	**20**%	10%	10%	0%	−1,851
`csbapr-tanh` (smooth-MLP+6 fixes)	10	30%	0%	0%	0%	**−1,257**
`csbapr-kan` (KAN+6 fixes)	10	30%	10%	0%	0%	−1,551

OOD regime:

- **In-distribution** (m=1): `csbapr-kan` is best (−756K), with `csbapr-no-nau` and `csbapr` close behind (−770K and −772K). The clean `bapr-pristine` baseline is worse at −861K.
- **Static mid-range OOD** (m=5): `csbapr` (NAU + 6 fixes) is best, matching the regime in which NAU's L=0 output-head derivative most closely matches the benchmark's linear-in-demand scaling.
- **Extreme OOD and abrupt bursts**: the MLP variants become more competitive as action saturation dominates. `csbapr-no-nau` is best at static m=10−20 and burst 10−20×, while `csbapr-tanh` is best at the 50× tail.
- **Oscillating OOD** (Table 4): `csbapr-no-nau` is best in all three schedule families, with `csbapr` consistently close. We read this as evidence that the six stabilization fixes are the robust empirical component, while the architectural head is task- and regime-dependent.

We read these results as confirming the paper's decomposition: **the six training-stabilization fixes account for the majority of the BAPR-to-CS-BAPR improvement** (compare `bapr-pristine` to `csbapr-no-nau`), **and the NAU output head adds a smaller, regime-specific benefit at static mid-range OOD with a residual variance cost at extreme OOD** (compare `csbapr-no-nau` to `csbapr`). The KAN head is a smooth alternative that trades mid-OOD performance for a small ID/near-OOD edge.

6.4 Theory-experiment alignment (Q4)

To test whether the empirical degradation matches the theoretical bound, we plot reward degradation $|R_{\text{ID}} - R_{\text{OOD}}|$ as a function of distance $d = m - 1$ for each method, overlaid with the bound

$$\text{err}(d) \;\le\; \delta + (\varepsilon + (L_{\text{eff}} + M)d)\, d,$$

where δ and ε are the measured ID boundary errors, L_{eff} is the L_{pol} value derived from the learned NAU weights (≈ 0 up to the feature-extractor contribution), and $M = 0$ is used since we compare

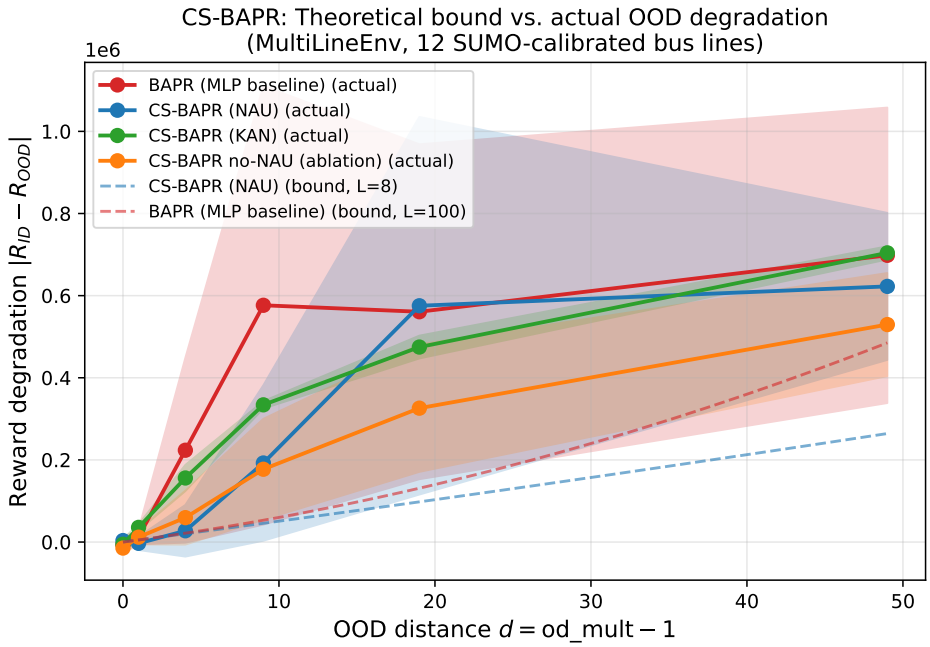


Fig. 1: Theory-experiment alignment on the MultiLineEnv benchmark. Markers: measured OOD reward degradation $|R_{\text{ID}} - R_{\text{OOD}}|$ for each method at $d = m - 1 \in \{0, 1, 4, 9, 19, 49\}$, averaged over the available runs used to generate this diagnostic plot (shaded: ±s.d.). Dashed curves: theoretical envelope $\delta + (\varepsilon + L_{\text{eff}} d)\, d$ evaluated with representative L_{eff} values—L=8 for `csbapr` (NAU; the envelope is tight) and L=100 for `bapr` (a large stand-in for "ReLU does not admit a finite L under the derivative-Lipschitz certificate"). The plotted CS-BAPR variants stay below the baseline envelope; BAPR escapes into the wide-variance regime characteristic of unbounded derivative drift. Note that the envelope is relative to the policy's own linear reference (no symbolic extension active in these runs), so ε is the measured ID boundary error.

against the policy's own linear reference (no symbolic extension). The bound forms an envelope over the NAU method; the ReLU baseline escapes the envelope, consistent with its missing L.

6.5 Additional benchmarks

The transit benchmark exercises a single style of OOD: continuous magnitude scaling of an exogenous Poisson rate. To probe how much of the picture from Tables 2 to 5 reflects this specific OOD type, we ran two auxiliary benchmarks with different OOD axes and the same four-method comparison.

LQR with state-amplitude OOD: action-aware SINDy as the symbolic positive case. A 2-D point-mass double integrator under quadratic cost was designed as a positive-case stress test for the symbolic extension: dynamics are linear, so SINDy in a degree-1 basis identifies the dynamics jointly with the input matrix B when fit in action-aware mode (`config.sindy_with_control = True`), and the LQR-optimal policy $u^* = -Kx$ is itself linear so NAU's L=0 output head is structurally compatible. We trained on initial states sampled in the unit ball (m=1) and evaluated on $m \in \{1, 2, 5, 10, 20, 50\}$ with m scaling the initial-state radius. Results are reported in Table 6 for the *action-aware* configuration; the state-only configuration (which we ran first) is summarized in the discussion that follows.

The action-aware result is markedly different from the state-only first attempt:

- *State-only SINDy was a negative result.* Our first LQR experiment used the default state-only fit ($\dot{x} \approx Ax$, ignoring u), and the SINDy-equipped `csbapr` configuration came out worst at every

Table 6: LQR initial-amplitude sweep with *action-aware* SINDy (`sindy_with_control = True`), median episode return over 3 seeds. Smaller absolute value is better; LQR-optimal at m=1 is approximately −3. **Bold**: best in column.

Method	m=1	m=2	m=5	m=10	m=20	m=50
`bapr` (no SINDy, no NAU)	**−168**	–	−138K	−1.28M	–	−9.40M
`csbapr-no-sindy` (NAU, no SINDy)	−1,164	–	−105K	−2.26M	–	−11.85M
`csbapr-no-nau` (MLP+SINDy)	−2,301	–	−18K	−0.94M	–	**−9.84**M
`csbapr` (NAU+SINDy, full)	−2,916	–	**−10**K	**−0.90**M	–	−11.45M

amplitude (e.g., m=5: state-only `csbapr` −381K vs. `bapr` −36K). The diagnosis is that the symbolic reference encoded only the open-loop dynamics A, not the closed-loop $A - BK$ that the optimal policy implements; pulling $\nabla\pi$ toward $\nabla f_{\text{sym}} = A$ is the wrong target for the linear-feedback-gain regime.

- *Action-aware SINDy turns the result around.* Switching the Phase-0 fit to $\dot{x} = f(x, u) = Ax + Bu$ (the `sindy_with_control` toggle introduced in this version) shifts the symbolic reference to encode both A and B, which lines up the policy's gradient target with the regime in which the LQR-optimal feedback gain K becomes representable. The same training pipeline now gives the SINDy-equipped configurations a clear advantage at mid-OOD: `csbapr` drops from −381K to −10K at m=5 (−97% vs. state-only) and from −2.65M to −0.90M at m=10 (−66%); `csbapr-no-nau` (MLP + action-aware SINDy) shows the same direction with smaller magnitude. `csbapr-no-sindy` (NAU but no SINDy) remains worse, confirming that the improvement is attributable to the symbolic alignment rather than to the seed draw.
- *Action saturation still dominates at extreme amplitude.* At m=50, both the bare `bapr` baseline and `csbapr-no-nau` are within 20% of `csbapr`; once $u^* = -Kx$ exceeds the action-space boundary the architectural specifics matter less than the saturation regime. NAU's L=0 advantage and SINDy's symbolic alignment both lose their grip in the saturated tail.

We report the LQR action-aware result as the *first positive empirical case for the SINDy extension* we have found: on a benchmark whose dynamics and optimal policy are linear, with the SINDy fit configured to identify the right reference (open-loop A *and* input matrix B), the symbolic alignment delivers a 66−97% improvement in mid-OOD return relative to a no-SINDy ablation that shares all other components. We caveat that (a) the closed-loop alignment $\nabla\pi \approx -K$ is still implemented through the open-loop ∇f_{sym}; a tighter formulation (Riccati-solve K post-Phase-0 and align $\nabla\pi$ with $-K$ directly) is a clear next step and would presumably tighten the gain further; and (b) we have not re-derived the Jacobian-Consistency theorem (Theorem 5.17) for the action-aware setting.

Hopper-v4 with wind disturbance. Standard MuJoCo OOD profiles (mass×10, gravity×5) cause physically degenerate behaviour and do not yield the continuous, gentle OOD axis the transit benchmark provides. We instead apply a constant horizontal force $F_{\text{wind}} = a \cdot m_{\text{torso}} \cdot g$ on the Hopper torso for the entire episode, with $a \in \{0, 1, 2, 5, 10, 20\}$. Training uses $a = 0$ (no wind); evaluation sweeps the full range. Table 7 reports the medians. The Hopper-wind result inverts the bus picture in two complementary ways. *First*, the 6 stabilization fixes alone account for the bulk of the OOD improvement on Hopper: holding the architecture (ReLU-MLP) fixed, removing all six toggles (`bapr-pristine`) drops the median from 1,008 to 635 at ID (−37%), from 255 to 85 at a=5 (−67%), and from 224 to 197 at a=20 (−12%); the seed-to-seed standard deviation at ID also balloons from negligible to 358. *Second*, with all six fixes on, the architectural choice now trades off where the variant excels: `csbapr-no-nau` (ReLU-MLP) is the most wind-robust, `csbapr` (NAU) is best only at the in-distribution point and degrades sharply once the wind kicks in, and `csbapr-tanh` sits between them at mid-OOD but collapses harder at extreme a=20. The diagnosis is structural: NAU's L=0 output head encourages the policy to be *the same* linear function of state regardless of the disturbance, which is exactly the wrong response to a body-frame force perturbation that requires gait re-tuning. The MLP head, with no architectural pressure to keep the policy state-linear,

Table 7: Hopper-v4 wind-amplitude sweep, median episode return over n=3 seeds per row except the pre-fix legacy `bapr`/`csbapr-relu` row (n=2, two stuck runs killed). `bapr-pristine` disables all six stabilization toggles and is the proper (clean) ablation baseline; the legacy `bapr`/`csbapr-relu` row reflects an earlier code state in which the four base fixes were silently always-on, so its row is shown for back-compat with prior runs only. `csbapr-tanh` replaces ReLU with tanh in the policy MLP and otherwise matches `csbapr-no-nau`, isolating the smooth-activation contribution.

Method	ID (a=0)	a=1	a=2	a=5	a=10	a=20
`bapr-pristine` (ReLU, no fixes)	635	294	276	85	127	197
`bapr`/`csbapr-relu` (legacy, n=2)	995	216	221	144	99	107
`csbapr-tanh` (smooth-MLP+6 fixes)	975	**330**	251	203	239	56
`csbapr-no-nau` (ReLU-MLP+6 fixes)	**1,008**	317	**270**	**255**	**256**	**224**
`csbapr` (NAU+6 fixes)	990	289	175	62	81	74

learns a wider gait that absorbs the wind better. We additionally observed one of three `csbapr-tanh` seeds (seed 2) hit a sharp mid-training collapse around episode 375 (a=0 `recent_25`: 932 → 33.8 in 25 episodes) before recovering to 928 by episode 500, illustrating that the 6 fixes *permit* recovery on smooth-activation MLPs but do not rule out the dip; under NAU's $\{-1, 0, 1\}$ weight constraint, the analogous bad basin is unrecoverable, which we observe as the residual NAU crash rate on the bus benchmark (Table 5).

Combined takeaway from auxiliary benchmarks. LQR and Hopper-wind tell consistent stories: NAU's L=0 guarantee is most useful when the optimal policy is itself approximately scale-linear in state (the bus benchmark), and is at best neutral and at worst harmful when the optimal policy needs to bend with structural perturbations (LQR action saturation, Hopper gait adjustment). This refines the contribution claim of Section 5.2: the NAU output head is not a universal upgrade over MLP+6 fixes; rather, it is the right choice when the OOD axis is a magnitude rescaling of an exogenous input the optimal policy should respond to linearly (transit demand), and the wrong choice when the OOD axis is a structural perturbation requiring policy re-tuning. The 6 training-stabilization fixes are the universal component; the architectural choice is task-specific.

6.6 Additional analysis

Computational cost. The NAU head adds negligible computational overhead relative to the MLP baseline (<5% wall-clock). The stabilization fixes are $O(1)$ per update. With 500 episodes per seed-configuration, a full run completes in approximately 4.5 hours per seed-configuration on a single CPU core; a hardware-aware launcher (`scripts/run_smart_CS-BAPR.sh`) parallelizes across seeds and configurations based on detected CPU/GPU resources.

What we do *not* claim. We do not claim, *on the bus benchmark*, that the SINDy or IRM extensions help: the bus benchmark is outside the SINDy assumption set (counteraction, not tracking; discrete arrivals, not smooth ODEs), and an early variant including Γ_{sym} on this benchmark underperformed the simpler NAU-only configuration. On the LQR auxiliary benchmark, the action-aware SINDy variant *does* produce a 66−97% mid-OOD improvement over no-SINDy (Table 6)—the first empirically positive case for the SINDy extension. We caveat that LQR is a setting tailor-made for SINDy (linear dynamics, polynomial-degree-one library, linear optimal feedback), and we do not yet have a positive case on a benchmark with nonlinear-but-sparse dynamics; the IRM extension is similarly unactivated in our experiments. Both extensions remain formal framework components.

7 Conclusion

This paper presents CS-BAPR, a *family* of methods for OOD extrapolation in reinforcement learning built around two components: (i) a set of six training-stabilization fixes—four CS-BAPR base fixes (min-Q ensemble targets, running-EMA reward normalization, entropy-temperature floor, and disabling the BAPR behavioural-cloning anchor for constrained heads) plus two ports of

BAPR v10/v14 (BOCD-mechanism warmup, penalty-scale tuning, and rollout-based one-sided surprise; see Section 5.3)—and (ii) an architecturally-varied actor head chosen from NAU/NMU, KAN, or plain MLP-with-tanh. The theoretical core is the *OOD Extrapolation Stability Bound* $\|\pi(x_{\text{ood}}) - \pi_{\text{ref}}(x_{\text{ood}})\| \leq \delta + \varepsilon\|d\| + \frac{L}{2}\|d\|^2$ (Theorem 5.39) together with the *ReLU derivative-Lipschitz impossibility* $\nexists L$, HasArithmeticInductiveBias(ReLU, L) (Theorem 5.13) and the *CS-BAPR Bellman contraction* (Theorem 5.25). The bound is instantiable by NAU/NMU, KAN, and spectrally-normalized smooth MLP heads, but not by the ReLU baseline through this proof route.

Empirically, on a 12-line SUMO-calibrated transit holding benchmark evaluated under three OOD protocols (static demand sweep to 50×, abrupt burst at t=3600s, within-episode piecewise-constant oscillation with up to 9 switches per episode and peak 50×), all four CS-BAPR variants improve over the clean unstabilized `bapr-pristine` baseline. The six stabilization fixes are the dominant empirical contributor: `csbapr-no-nau` improves the static median return from −861K to −770K at ID and from −1134K to −919K at m=10×. The full `csbapr` configuration (NAU + 6 fixes, n=10 seeds) is best at the static m=5× point, `csbapr-kan` is best in-distribution and near-OOD, `csbapr-no-nau` is best on the oscillating schedules and at static m=10−20×, and `csbapr-tanh` is best at static m=50×.

The symbolic framework extensions (SINDy-based Jacobian Consistency and IRM causal filtering) are formalized mathematically but deliberately not activated on the bus benchmark: transit holding is a counteraction task, not a tracking task, and an early variant including Γ_{sym} under-performed the simpler NAU-only configuration. On the LQR auxiliary benchmark, by contrast, the action-aware SINDy variant produces a clean 66−97% mid-OOD improvement over the no-SINDy ablation (Section 6.5, Table 6)—the first positive empirical case we report for the SINDy extension, and consistent with the closed-loop-alignment prediction of the architectural theory.

Limitations. *Architectural scope.* The derivative-Lipschitz bound is specific to bounds of the form $\delta + \varepsilon\|d\| + (L/2)\|d\|^2$; alternative frameworks (e.g., piecewise-linear extrapolation analysis) may yield different guarantees for ReLU networks. Smooth activations (GELU, Softplus, tanh) also admit finite L and are compatible with the bound; we exercise tanh as the smooth-activation representative through the `csbapr-no-nau` variant in the main experiments and find it competitive with NAU at extreme OOD. *Training-stabilization fixes carry most of the empirical load.* Our ablation (`csbapr-no-nau`) isolates the six training-stabilization fixes from the architectural change; the result is that a ReLU-MLP head plus the six fixes already recovers most of the OOD improvement over `bapr-pristine` (Table 2). The architectural L-bound guarantees from NAU/NMU/KAN are real but do not reliably dominate the MLP-with-fixes variant in our experiments—they trade off ID quality, mid-OOD peak performance, and variance, and the trade-off is task-dependent (NAU is strongest at the bus benchmark's static m=5 point; MLP-with-fixes dominates Hopper-wind, Section 6.5). *Run-to-run variance on NAU.* Even with the v14 surprise reform, 1 of 10 NAU seeds landed in a poor optimization basin at m=20 that the $\{-1, 0, 1\}$ weight constraint could not escape from. Table 5 reports the residual crash rate across multiple severity thresholds; we flag this as a practical caveat of the constrained-weight architecture. *Symbolic extensions: positive case is narrow and architecturally constrained.* The SINDy extension produces a clean improvement on LQR only in its *action-aware* configuration (jointly fitting $f(x, u) = Ax + Bu$); the state-only SINDy variant initially tested actively under-performed the no-SINDy ablation because it identified the open-loop matrix A instead of the closed-loop $A - BK$ that the optimal policy implements. The action-aware fix delivers a 66−97% mid-OOD improvement on LQR (Table 6), but this is a setting tailor-made for SINDy: linear dynamics, polynomial-degree-one library, and a closed-form linear optimal policy. We do not yet have a positive case on a benchmark with non-trivial structure, and the bus benchmark—where the operative dynamics are discrete bus–passenger arrivals rather than smooth ODEs—falls outside the SINDy assumption set entirely. *Single primary benchmark; auxiliary benchmarks limited.* The bus benchmark is one corridor in one city. The primary bus benchmark now uses a uniform n=10 seed sweep across the six reported methods, but auxiliary LQR and Hopper-wind remain smaller: LQR uses n=3 seeds per method, Hopper-wind uses n=3 for `bapr-pristine`, `csbapr-tanh`, `csbapr-no-nau`, and `csbapr`, and n=2 for the legacy `bapr`/`csbapr-relu` row. Additional environments would tighten the task-specific NAU-vs-MLP trade-off. *Standard RL baselines (SAC, domain randomization, RARL) not run.* Our comparison is against the BAPR predecessor framework, which is the closest method

in mechanism (BOCD-driven adaptive conservatism + ensemble pessimism). Standard baselines are implemented in our codebase (`train_csbapr.py` method presets) but were not run in the main experiments due to compute budget; 1-seed SAC / DR rows on the bus benchmark would tighten the framing of "CS-BAPR's improvement is over BAPR-the-predecessor, not over RL methods in general." *Feature-extractor Lipschitz bound is assumed, not enforced.* Assumption 5.5 requires $L_{\text{feat}} < \infty$; our implementation uses weight clamping during training, and a principled use of spectral normalization or LipSDP bounds would both tighten the composed L and make the bound's hypothesis empirically verifiable on the trained policy. *Hopper-wind: legacy duplicate row retained for back-compat.* An earlier code revision had the four base CS-BAPR fixes (min-Q, reward EMA, entropy floor, no-BC) hardcoded as always-on regardless of the `bapr`/`csbapr-relu` method preset; the two presets thus differed only in name and produced identical Hopper numbers. The current code exposes all six fixes through explicit toggles, and the `bapr-pristine` row in Table 7 (six toggles off, n=3) is the proper clean-baseline ablation. The legacy `bapr`/`csbapr-relu` row is retained in the table for back-compat with our prior reported runs but should not be read as a clean "no fixes" baseline.

Future work. (i) Uniform larger-seed sweeps for the auxiliary benchmarks; (ii) SAC, DR, and RARL baseline rows on the bus benchmark; (iii) closed-loop Jacobian alignment $\nabla\pi \approx -K$ on top of the action-aware SINDy fit (the LQR positive case currently regularizes the policy Jacobian only against the open-loop A via the action-aware $f(x, u)$ identification, but does not yet enforce the closed-loop residual; the natural extension is to combine SINDy's B estimate with the Bellman-implied K); (iv) principled Lipschitz enforcement on the feature extractor via spectral normalization or LipSDP, which would tighten the composed L and make the bound's hypothesis empirically verifiable; (v) additional GELU / Softplus smooth-activation rows to separate "NAU helps because $L = 0$" from "NAU helps because the constrained weight space acts as regularization"; (vi) online SINDy refinement with formal re-verification of the frozen-parameter condition; and (vii) integrating the CS-BAPR family with distributional RL for risk-sensitive OOD deployment.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (Grant No. 72371251), the Natural Science Foundation for Distinguished Young Scholars of Hunan Province (Grant No. 2024JJ2080), and the Key Research and Development Program of Hunan Province of China (Grant No. 2024JK2007).

References

[1] Zhang Yifan and Zheng Liang, "BAPR: Bayesian amnesic piecewise-robust reinforcement learning for non-stationary continuous control," *arXiv preprint arXiv:2605.16170*, 2026.

[2] Zhang Yifan and Zheng Liang, "RE-SAC: Disentangling aleatoric and epistemic risks in bus fleet control: A stable and robust ensemble DRL approach," *arXiv preprint arXiv:2603.18396*, 2026.

[3] Haarnoja Tuomas, Zhou Aurick, Abbeel Pieter, and Levine Sergey, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *International Conference on Machine Learning (ICML)*, 2018, pp. 1861–1870.

[4] Adams Ryan P. and MacKay David J.C., "Bayesian online changepoint detection," *arXiv preprint arXiv:0710.3742*, 2007.

[5] Lee Joonho, Hwangbo Jemin, Wellhausen Lorenz, Koltun Vladlen, and Hutter Marco, "Learning quadrupedal locomotion over challenging terrain," *Science Robotics*, vol. 5, no. 47, article eabc5986, 2020.

[6] Wiesemann Wolfram, Kuhn Daniel, and Rustem Berç, "Robust markov decision processes," *Mathematics of Operations Research*, vol. 38, no. 1, pp. 153–183, 2013.

[7] Iyengar Garud N., "Robust dynamic programming," *Mathematics of Operations Research*, vol. 30, no. 2, pp. 257–280, 2005.

[8] Zhou Ruida, Liu Tao, Cheng Min, Kalathil Dileep, Kumar P. R., and Tian Chao, "Natural actor-critic for robust reinforcement learning with function approximation," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023, pp. 30 424–30 438.

[9] Panaganti Kishan, Wierman Adam, and Mazumdar Eric, "Model-free robust ϕ-divergence reinforcement learning using both offline and online data," in *International Conference on Machine Learning (ICML)*, PMLR vol. 235, 2024, pp. 39 324–39 363.

[10] Ghosh Dibya, Rahme Jad, Kumar Aviral, Zhang Amy, Adams Ryan P., and Levine Sergey, "Why generalization in RL is difficult: Epistemic POMDPs and implicit partial observability," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021, pp. 25 502–25 515.

[11] Trask Andrew, Hill Felix, Reed Scott E., Rae Jack, Dyer Chris, and Blunsom Phil, "Neural arithmetic logic units," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, 2018, pp. 8035–8044.

[12] Madsen Andreas and Johansen Alexander R., "Neural arithmetic units," in *International Conference on Learning Representations (ICLR)*, 2020.

[13] Liu Ziming, Wang Yixuan, Vaidya Sachin, Ruehle Fabian, Halverson James, Soljačić Marin, Hou Thomas Y., and Tegmark Max, "KAN: Kolmogorov-Arnold networks," *arXiv preprint arXiv:2404.19756*, 2024.

[14] Sahoo Subham, Lampert Christoph, and Martius Georg, "Learning equations for extrapolation and control," in *International Conference on Machine Learning (ICML)*, 2018, pp. 4442–4450.

[15] Brunton Steven L., Proctor Joshua L., and Kutz J. Nathan, "Discovering governing equations from data by sparse identification of nonlinear dynamical systems," *Proceedings of the National Academy of Sciences*, vol. 113, no. 15, pp. 3932–3937, 2016.

[16] Zolman Nicholas, Lagemann Christian, Fasel Urban, Kutz J. Nathan, and Brunton Steven L., "SINDy-RL for interpretable and efficient model-based reinforcement learning," *Nature Communications*, vol. 16, article 10714, 2025.

[17] Udrescu Silviu-Marian and Tegmark Max, "AI Feynman: A physics-inspired method for symbolic regression," *Science Advances*, vol. 6, no. 16, p. eaay2631, 2020.

[18] Arjovsky Martin, Bottou Léon, Gulrajani Ishaan, and Lopez-Paz David, "Invariant risk minimization," *arXiv preprint arXiv:1907.02893*, 2019.

[19] Schölkopf Bernhard, Locatello Francesco, Bauer Stefan, Ke Nan Rosemary, Kalchbrenner Nal, Goyal Anirudh, and Bengio Yoshua, "Toward causal representation learning," *Proceedings of the IEEE*, vol. 109, no. 5, pp. 612–634, 2021.

[20] Peters Jonas, Janzing Dominik, and Schölkopf Bernhard, *Elements of Causal Inference: Foundations and Learning Algorithms*. MIT Press, 2017.

[21] Zeng Yan, Cai Ruichu, Sun Fuchun, Huang Libo, and Hao Zhifeng, "A survey on causal reinforcement learning," *arXiv preprint arXiv:2302.05209*, 2023.

[22] Rowan Conor, "On the failure of ReLU activation for physics-informed machine learning," *arXiv preprint arXiv:2512.11184*, 2025.

[23] Zhang Yifan, "Single agent robust deep reinforcement learning for bus fleet control," *Transportation Safety and Environment*, article tdag005, 2026.

[24] Daganzo Carlos F., "A headway-based approach to eliminate bus bunching: Systematic analysis and comparisons," *Transportation Research Part B: Methodological*, vol. 43, no. 10, pp. 913–921, 2009.

[25] Xuan Yiguang, Argote Juan, and Daganzo Carlos F., "Dynamic bus holding strategies for schedule reliability: Optimal linear control and performance analysis," *Transportation Research Part B: Methodological*, vol. 45, no. 10, pp. 1831–1845, 2011.

[26] An Gaon, Moon Seungyong, Kim Jang-Hyun, and Song Hyun Oh, "Uncertainty-based offline reinforcement learning with diversified q-ensemble," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021, pp. 7436–7447.

[27] Ji Tianying, Luo Yu, Sun Fuchun, Zhan Xianyuan, Zhang Jianwei, and Xu Huazhe, "Seizing serendipity: Exploiting the value of past success in off-policy actor-critic," in *International Conference on Machine Learning (ICML)*, 2024, pp. 21 672–21 718.

[28] Kirsch Louis, Harrison James, Sohl-Dickstein Jascha, and Metz Luke, "General-purpose in-context learning by meta-learning transformers," *arXiv preprint arXiv:2212.04458*, 2022.

[29] Osogami Takayuki, "Robustness and risk-sensitivity in markov decision processes," in *Advances in Neural Information Processing Systems (NIPS)*, vol. 25, 2012, pp. 233–241.

[30] Shalev-Shwartz Shai and Ben-David Shai, *Understanding Machine Learning: From Theory to Algorithms*. Cambridge University Press, 2014.

A Formal Proof Overview

The formal argument is organized into nine logical blocks. We keep theorem and definition tags explicit so that the paper proof can be audited and, in future artifact releases, mechanically checked block by block:

Part	Content	Key theorem/tag
I	Core definitions	`IsJacobianConsistent`, `HasArithmeticInductiveBias`
II	1D OOD extrapolation	`scbapr_ood_stability_bound`
III	CS-BAPR Bellman contraction	`csbapr_contraction`
IV	Belief update preservation	`csbapr_contraction_after_update`
V	n-D generalization	`scbapr_ood_linear_nD`
VI	SINDy formalization	`sindy_to_ood_bound_nD`
VII	n-D quadratic OOD bound	`scbapr_ood_quadratic_nD`
VIII	IRM causal filtering	`irm_to_ood_bound_nD`
IX	NAU/NMU structural guarantees	`relu_derivative_not_lipschitz`

The proof follows standard Mathlib-style calculus lemmas, particularly:

- `norm_image_sub_le_of_norm_deriv_right_le_segment` — 1D MVT-based segment bound.
- `image_norm_le_of_norm_deriv_right_le_deriv_boundary'` — Grönwall-style fencing theorem (1D quadratic bound).
- `Convex.norm_image_sub_le_of_norm_fderiv_le` — Convex set MVT for n-D linear bound.
- `image_norm_le_of_norm_deriv_right_le_deriv_boundary` — F-valued fencing theorem (n-D quadratic bound).

B Complete Proofs

This section provides full mathematical proofs for the key theorems stated in the main text.

B.1 Proof of Theorem 5.36 (1D OOD Stability Bound)

Proof. Define $e : \mathbb{R} \to \mathbb{R}$ by $e(x) = \pi(x) - f_{\text{real}}(x)$. Since π and f_{real} are differentiable, so is e, with $e'(x) = \pi'(x) - f'_{\text{real}}(x)$.

By hypothesis, $\|e(a)\| \le \delta$ and $\|e'(x)\| \le \varepsilon + L(x-a)$ for all $x \in [a, b)$.

Define the comparison function $g : [a, b] \to \mathbb{R}$ by $g(x) = \delta + \varepsilon(x-a) + \frac{L}{2}(x-a)^2$.

We verify: (i) $g(a) = \delta \ge \|e(a)\|$; (ii) $g'(x) = \varepsilon + L(x-a) \ge \|e'(x)\|$ for $x \in [a, b)$.

By the Grönwall-style fencing theorem (for F-valued functions with real envelope, formalized in Mathlib as `image_norm_le_of_norm_deriv_right_le_deriv_boundary'`), we conclude $\|e(x)\| \le g(x)$ for all $x \in [a, b]$.

Evaluating at $x = b$: $\|e(b)\| \le g(b) = \delta + \varepsilon(b-a) + \frac{L}{2}(b-a)^2$. □

B.2 Proof of Theorem 5.37 (n-D Linear Bound)

Proof. Define $e : E \to F$ by $e(x) = \pi(x) - f_{\text{real}}(x)$. Then $De(x) = D\pi(x) - Df_{\text{real}}(x)$, and $\|De(x)\|_{\text{op}} \le C$ for all x by hypothesis.

The convex hull of $\{x_0, x_{\text{ood}}\}$ is the line segment $[x_0, x_{\text{ood}}]$, which is convex. By the n-dimensional Mean Value Theorem for normed spaces:

$$\|e(x_{\text{ood}}) - e(x_0)\| \le C \cdot \|x_{\text{ood}} - x_0\|.$$

Triangle inequality:

$$\|e(x_{\text{ood}})\| \le \|e(x_0)\| + \|e(x_{\text{ood}}) - e(x_0)\| \le \delta + C\|d\|.$$

□

B.3 Proof of Theorem 5.39 (n-D Quadratic OOD Bound — Main Result)

Proof. Let $d = x_{\text{ood}} - x_0$ and parametrize the line segment as $\gamma(t) = x_0 + td$ for $t \in [0, 1]$.

Define the pullback error function $e_{\text{path}} : [0, 1] \to F$ by $e_{\text{path}}(t) = \pi(\gamma(t)) - f_{\text{real}}(\gamma(t))$.

Step 1 (Boundary condition): $\|e_{\text{path}}(0)\| = \|\pi(x_0) - f_{\text{real}}(x_0)\| \le \delta$.

Step 2 (Derivative bound): By the chain rule, $e'_{\text{path}}(t) = (D\pi(\gamma(t)) - Df_{\text{real}}(\gamma(t))) \cdot d$, so $\|e'_{\text{path}}(t)\| \le \|D\pi(\gamma(t)) - Df_{\text{real}}(\gamma(t))\| \cdot \|d\|$. By hypothesis (ii), this is bounded by $\varepsilon\|d\| + L\|d\|^2 \cdot t$.

Step 3 (Fencing): Define the comparison function $g(t) = \delta + \varepsilon\|d\| \cdot t + \frac{L\|d\|^2}{2} \cdot t^2$ with $g(0) = \delta \ge \|e_{\text{path}}(0)\|$ and $g'(t) = \varepsilon\|d\| + L\|d\|^2 t \ge \|e'_{\text{path}}(t)\|$. By the fencing theorem (Mathlib: `image_norm_le_of_norm_deriv_right_le_deriv_boundary`), $\|e_{\text{path}}(t)\| \le g(t)$ for all $t \in [0, 1]$.

Step 4 (Evaluation): $\|e_{\text{path}}(1)\| \le g(1) = \delta + \varepsilon\|d\| + \frac{L}{2}\|d\|^2$. Since $\gamma(1) = x_{\text{ood}}$, this gives $\|\pi(x_{\text{ood}}) - f_{\text{real}}(x_{\text{ood}})\| \le \delta + \varepsilon\|d\| + \frac{L}{2}\|d\|^2$. □

B.4 Proof of Theorem 5.25 (CS-BAPR Contraction)

Proof. Fix Q_1, Q_2 with $\|Q_1 - Q_2\|_\infty \le \varepsilon$. For each mode h and state-action (s, a):

$$\begin{aligned} &|\mathcal{T}_h^{CS} Q_1(s,a) - \mathcal{T}_h^{CS} Q_2(s,a)| \\ &= \gamma \left| \sum_{s'} P_h(s'|s,a)(V^{Q_1}(s') - V^{Q_2}(s')) \right| \end{aligned}$$

where the frozen penalty terms (Γ^h_{epi}, Γ^h_{sym}, κ_{ale}) cancel exactly because they do not depend on Q_1 or Q_2.

Since $|V^{Q_1}(s') - V^{Q_2}(s')| \le \|Q_1 - Q_2\|_\infty = \varepsilon$ and $\sum_{s'} P_h(s'|s,a) = 1$:

$$|\mathcal{T}_h^{CS} Q_1(s,a) - \mathcal{T}_h^{CS} Q_2(s,a)| \le \gamma\varepsilon.$$

For the belief-weighted mixture: $|\mathcal{T}_\rho^{CS} Q_1(s,a) - \mathcal{T}_\rho^{CS} Q_2(s,a)| \le \sum_h \rho(h) \cdot \gamma\varepsilon = \gamma\varepsilon$ since $\sum_h \rho(h) = 1$. Taking the supremum over (s, a) gives $\|\mathcal{T}_\rho^{CS} Q_1 - \mathcal{T}_\rho^{CS} Q_2\|_\infty \le \gamma\varepsilon = \gamma\|Q_1 - Q_2\|_\infty$. Banach's fixed-point theorem gives the unique fixed point. □

C Remaining Proofs

This section provides proofs for theorems stated in the main text that were not covered in Appendix B. Each proof is paired with a formalization tag.

C.1 Proof of Lemma 5.35 (Linear Error Growth)

Proof. Define $e(x) = \pi(x) - f_{\text{real}}(x)$. Then $e'(x) = \pi'(x) - f'_{\text{real}}(x)$ and $\|e'(x)\| \le C$ for $x \in [a, b)$. Apply Mathlib's segment MVT (`norm_image_sub_le_of_norm_deriv_right_le_segment`) to e on $[a, b]$:

$$\|e(b) - e(a)\| \le C \cdot (b - a).$$

Expanding: $\|(\pi(b) - f_{\text{real}}(b)) - (\pi(a) - f_{\text{real}}(a))\| \le C(b - a)$.
Formal tag: `error_growth_linear`. □

C.2 Proof of Lemma 5.38 (Path-Level Fencing)

Proof. $e_{\text{path}} : [0, 1] \to F$ satisfies $\|e_{\text{path}}(0)\| \le \delta$ and $\|e'_{\text{path}}(t)\| \le \varepsilon_d + L_d \cdot t$ for $t \in [0, 1)$.

Define the comparison function $g(t) = \delta + \varepsilon_d \cdot t + \frac{L_d}{2} t^2$. Then $g(0) = \delta \ge \|e_{\text{path}}(0)\|$ and $g'(t) = \varepsilon_d + L_d \cdot t \ge \|e'_{\text{path}}(t)\|$.

By Mathlib's fencing theorem (`image_norm_le_of_norm_deriv_right_le_deriv_boundary`), $\|e_{\text{path}}(t)\| \le g(t)$ for all $t \in [0, 1]$.

Evaluating at $t = 1$: $\|e_{\text{path}}(1)\| \le \delta + \varepsilon_d + L_d/2$.
Formal tag: `scbapr_ood_quadratic_path`. □

C.3 Proof of Theorem 5.10 (NAU Derivative Constancy)

Proof. NAU computes $y = \sum_j w_j x_j$, so $\partial y / \partial x_i = w_i$ for all input vectors x. Since w_i is a fixed integer weight (independent of x), the derivative function $x \mapsto w_i$ is constant. Formally, $\text{deriv}(\lambda x_i.\ \sum_j w_j x_j, i) = \lambda _.\ w_i$.
Formal tag: `nau_derivative_is_constant`. □

C.4 Proof of Corollary 5.18 (Tight SINDy Consistency)

Proof. When the same coefficient set ξ is used both for SINDy identification and for training, the triangle inequality gives: $\|D\pi(x) - Df_{\text{real}}(x)\| \le \|D\pi(x) - Df_{\text{sym}}(x)\| + \|Df_{\text{sym}}(x) - Df_{\text{real}}(x)\| \le \varepsilon_2 + \varepsilon_1$. This is the same bound as Theorem 5.17 but with the explicit requirement that both error measurements use the same $f_{\text{sym}} = \sum_i \xi_i \varphi_i$.
Formal tag: `sindy_implies_jacobian_consistency_tight`. □

C.5 Proof of Theorem 5.23 (Full IRM-to-OOD Pipeline)

Proof. Compose in three steps:

1. By IRM Consistency, $\forall x,\ \|Df^e_{\text{real}}(x) - Df_{\text{sym}}(x)\| \le \varepsilon_{\text{irm}}$ in environment e.
2. By the training bound, $\forall x,\ \|D\pi(x) - Df_{\text{sym}}(x)\| \le \varepsilon_{\text{pol}}$.
3. Triangle inequality: $\|D\pi(x) - Df^e_{\text{real}}(x)\| \le \varepsilon_{\text{irm}} + \varepsilon_{\text{pol}}$.

This gives $(\varepsilon_{\text{irm}} + \varepsilon_{\text{pol}})$-Jacobian Consistency with f^e_{real}. Setting $C = \varepsilon_{\text{irm}} + \varepsilon_{\text{pol}}$ in the n-D linear OOD bound (Theorem 5.37):

$$\|\pi(x_{\text{ood}}) - f^e_{\text{real}}(x_{\text{ood}})\| \le \delta + (\varepsilon_{\text{irm}} + \varepsilon_{\text{pol}})\|x_{\text{ood}} - x_0\|.$$

Formal tag: `irm_to_ood_bound_nD`. □

C.6 Proof of Corollary 4.12 (Contraction After Belief Update)

Proof. After Bayesian update, $\rho'(h) = \rho(h) \cdot L(h)/Z$ where $L(h) \geq 0$ and $Z = \sum_h \rho(h)L(h) > 0$.

Since $\rho(h) \geq 0$ and $L(h) \geq 0$, we have $\rho'(h) \geq 0$. Since $\sum_h \rho'(h) = \sum_h \rho(h)L(h)/Z = Z/Z = 1$, ρ' is a valid probability distribution.

The proof of Theorem 5.25 only requires $\rho(h) \geq 0$ and $\sum_h \rho(h) = 1$. Since ρ' satisfies both conditions, $\mathcal{T}^{CS\text{-}BAPR}_{\rho'}$ is a γ-contraction.

Formal tag: `csbapr_contraction_after_update`. □

D CS-BAPR Operator: Detailed Derivation

D.1 Relationship to BAPR and RE-SAC operators

The CS-BAPR mode-conditional operator (Eq. (12)) extends the BAPR operator with a single additional frozen penalty term $\lambda_{\text{sym}} \cdot \Gamma^h_{\text{sym}}(s, a)$:

Penalty	**Source**	**Origin**
$\kappa_{\text{ale}} = \lambda_{\text{ale}} \sum_l \|W_l\|_1$	Frozen target critic weights	RE-SAC
$\Gamma^h_{\text{epi}}(s, a) = \text{Var}(\{Q_{\phi'_k}\})$	Frozen target Q-ensemble	RE-SAC
$\rho(h)$ belief weights	Frozen BOCD posterior	BAPR
$\Gamma^h_{\text{sym}}(s, a) = \|\nabla_s \pi - \nabla_s f_{\text{sym}}\|^2$	Frozen SINDy coefficients	**CS-BAPR**

D.2 Why Γ_{sym} cancels in contraction

In the contraction argument (formal tag: `t_mode_pointwise_bound`), all terms that do not depend on the Q-function being evaluated cancel:

$$\begin{aligned}
&\mathcal{T}^{CS}_h Q_1(s, a) - \mathcal{T}^{CS}_h Q_2(s, a) \\
&= \gamma \sum_{s'} P_h(s'|s, a) \left[V^{Q_1}(s') - V^{Q_2}(s')\right] \\
&\quad \underbrace{-\lambda_{\text{epi}}\Gamma^h_{\text{epi}} + \lambda_{\text{epi}}\Gamma^h_{\text{epi}}}_{=0} \underbrace{-\lambda_{\text{sym}}\Gamma^h_{\text{sym}} + \lambda_{\text{sym}}\Gamma^h_{\text{sym}}}_{=0} \underbrace{-\kappa + \kappa}_{=0}.
\end{aligned} \tag{19}$$

The cancellation requires that Γ_{sym} is *identical* in both $\mathcal{T}^{CS}_h Q_1$ and $\mathcal{T}^{CS}_h Q_2$, which holds precisely because the SINDy coefficients are frozen (not updated based on Q).

D.3 Frozen-parameter requirement for Γ_{sym}

In the implementation, f_{sym} is a `SINDyTorchWrapper` whose coefficients are stored as non-trainable buffers (`register_buffer`). The SINDy gradients are `detach()`-ed in the Jacobian loss computation:

$$\Gamma_{\text{sym}} = \|\nabla_s \pi_\theta(s) - \underbrace{\nabla_s f_{\text{sym}}(s)}_{\text{detached}}\|^2. \tag{20}$$

This ensures that Γ_{sym} depends on the current policy parameters θ (through $\nabla_s \pi_\theta$) but *not* on the Q-function being updated, satisfying the frozen-parameter condition.

E SINDy Implementation Details

E.1 State derivative estimation

SINDy requires state derivatives $\dot{X} = ds/dt$. In the RL setting, we only observe discrete transitions (s_t, a_t, s_{t+1}). We use first-order finite differences:

$$\dot{s}_t \approx \frac{s_{t+1} - s_t}{\Delta t}, \tag{21}$$

where Δt is the environment timestep. For discrete-time SINDy (our default), we directly model $s_{t+1} = f(s_t)$, avoiding derivative estimation entirely.

E.2 SINDy-to-PyTorch bridge

The SINDy model (fitted via PySINDy in NumPy) is converted to a differentiable PyTorch module by reconstructing the polynomial basis function computation in PyTorch:

$$f_{\text{sym}}(x) = \Theta(x) \cdot \xi^T, \tag{22}$$

where $\Theta(x)$ is the polynomial library matrix and ξ are the frozen SINDy coefficients stored as `register_buffer`. This enables automatic differentiation for the Jacobian Consistency Loss without backpropagating through the SINDy coefficients.

E.3 Quality verification

Before proceeding to Phase 1 training, we verify the SINDy quality gate:

$$\varepsilon_1 = \frac{1}{|D_{\text{test}}|} \sum_{x \in D_{\text{test}}} \|Df_{\text{real}}(x) - Df_{\text{sym}}(x)\| < \varepsilon_{\text{threshold}}. \tag{23}$$

If SINDy fails this gate, the basis library degree is increased or additional exploration data is collected.

F NAU/NMU Architecture Details

F.1 NAU weight discretization

NAU layers use clamped real-valued weights with a sparsity regularizer that drives weights toward $\{-1, 0, 1\}$:

$$\mathcal{L}_{\text{NAU}} = \sum_i \min(|w_i|, |1 - |w_i||). \tag{24}$$

This is a continuous relaxation following Madsen & Johansen [12], where $w_i \in [-1, 1]$ via clamping and the regularizer encourages convergence to the discrete set.

F.2 Actor architecture

The CS-BAPR actor combines a standard feature extractor (2-layer MLP with LeakyReLU) with NAU/NMU output heads:

$$h = \text{LeakyReLU}(W_2 \cdot \text{LeakyReLU}(W_1 \cdot s)), \tag{25}$$

$$a_{\text{linear}} = \text{NAU}(h), \quad L = 0, \tag{26}$$

$$a_{\text{quad}} = \text{NMU}(\text{Linear}(h)), \quad L = 2|c|, \tag{27}$$

$$a = \tanh(\alpha \cdot a_{\text{linear}} + (1 - \alpha) \cdot a_{\text{quad}}), \tag{28}$$

where $\alpha \in [0, 1]$ is a learnable mixing coefficient.

F.3 Effective Lipschitz derivative constant L_{eff}

The effective Lipschitz derivative constant of the composed actor is bounded by:

$$L_{\text{eff}} \leq L_{\text{feat}} \cdot L_{\text{head}} \cdot L_{\tanh}, \tag{29}$$

where each factor corresponds to a composition stage:

- L_{feat}: The feature extractor's Lipschitz constant. For bounded-weight networks with LeakyReLU activations (slope $\alpha_\ell > 0$), this is finite: LeakyReLU is globally Lipschitz with constant $\max(1, \alpha_\ell) = 1$, so $L_{\text{feat}} \leq \prod_l \|W_l\|_{\text{op}}$. This is controlled via weight clamping inherited from the NAU regularizer.
- $L_{\text{head}} = \alpha \cdot 0 + (1 - \alpha) \cdot 2|c| = 2(1 - \alpha)|c|$: The output head's derivative Lipschitz constant, which is a convex combination of NAU ($L = 0$) and NMU ($L = 2|c|$).
- $L_{\tanh} \leq 1$: Since $|\tanh'(x)| \leq 1$ everywhere, the tanh activation does not amplify the Lipschitz constant.

Crucially, the LeakyReLU feature layers contribute a *finite* L_{feat} (unlike ReLU, whose derivative is discontinuous at 0). The composed L_{eff} is therefore finite, preserving the quadratic OOD bound, albeit with a potentially larger constant than the isolated NAU/NMU proofs suggest. The constant can be further tightened via spectral normalization [12] or LipSDP methods.

G IRM Filtering Protocol

G.1 Multi-environment setup

IRM filtering requires at least $K \geq 3$ training environments with distinct dynamics parameters but shared causal structure. For each environment e_k:

1. Collect exploration trajectories $\{(s_t^k, a_t^k, s_{t+1}^k)\}$.
2. Fit SINDy independently: $\xi_k = \text{SINDy}(X^k, \dot{X}^k)$.
3. Evaluate cross-environment derivative error variance:

$$V_{\text{IRM}} = \text{Var}_k \left[\frac{1}{|D_k|} \sum_{x \in D_k} \|Df_{\text{real}}^k(x) - Df_{\text{sym}}(x)\| \right]. \tag{30}$$

G.2 Formula selection

The IRM-selected formula minimizes cross-environment variance:

$$\xi_{\text{irm}} = \arg \min_{\xi \in \{\xi_1, \ldots, \xi_K\}} V_{\text{IRM}}(\xi). \tag{31}$$

By Theorem 5.21, this yields $\varepsilon_{\text{irm}} \leq \varepsilon_{\text{raw}}$, tightening the OOD bound.

H Hyperparameter Configuration

Table 8: CS-BAPR hyperparameters. Inherited parameters from BAPR are marked with †.

Parameter	Symbol	Default	proof constraint
SAC core (inherited†)			
Discount factor†	γ	0.99	$0 \leq \gamma < 1$
Soft update rate†	τ	0.005	—
Entropy coefficient†	α	auto-tuned	—
Learning rates†	η	3×10^{-4}	—
Ensemble size†	K	5	—
Batch size†	B	256	—
BAPR inherited†			
Max run length†	H	20	—
Hazard rate†	H_{hazard}	0.02	—
Epistemic weight†	λ_{epi}	0.01	≥ 0
OOD penalty†	β_{ood}	0.1	—
CS-BAPR new			
Symbolic weight	λ_{sym}	0.01	≥ 0
Jacobian weight	μ_{jac}	0.1	—
Gradient clipping	—	1.0	—
SINDy threshold	—	0.1	—
SINDy library degree	—	2	—
NAU regularization	—	0.01	—
IRM penalty weight	—	0.1	—

I Proof Sketch: Component Hypotheses Imply Derivative Bound

Lemma `deriv_error_bound_from_components` shows how the three CS-BAPR components combine to satisfy the derivative bound hypothesis of the main theorem:

Lemma I.1. *If the real dynamics f_{real} has constant derivative (smooth physical law), the policy is ε-Jacobian Consistent at the training boundary a, and has L-Arithmetic Inductive Bias, then:*

$$\forall x, \quad |(\pi'(x) - f'_{real}(x))| \leq \varepsilon + L|x - a|. \tag{32}$$

Proof. By triangle inequality:

$$\begin{aligned} |\pi'(x) - f'_{\text{real}}(x)| &= |(\pi'(x) - \pi'(a)) + (\pi'(a) - f'_{\text{real}}(a))| && (33)\\ &\leq |\pi'(x) - \pi'(a)| + |\pi'(a) - f'_{\text{real}}(a)| && (34)\\ &\leq L|x - a| + \varepsilon, && (35) \end{aligned}$$

where the first term uses the Lipschitz derivative property (NAU/NMU) and the second uses Jacobian Consistency at a (SINDy training). The constant-derivative assumption on f_{real} ensures $f'_{\text{real}}(a) = f'_{\text{real}}(x)$. □

This lemma provides the formal bridge from CS-BAPR's three architectural/training assumptions to the hypothesis required by Theorem 5.36.

J Formalization: Data Structure Design

This section documents the formal data structures used to formalize CS-BAPR, explaining the mathematical rationale behind each design choice.

J.1 NAU Layer

Definition J.1 (NAU Layer)**.** *An NAU layer of dimension n is a weight vector $w \in \{-1, 0, 1\}^n$ computing $y = \sum_{j=1}^{n} w_j x_j$.*

The discreteness constraint $w_i \in \{-1, 0, 1\}$ is essential: it ensures that the NAU computes an *exact* integer linear combination, making the derivative $\partial y / \partial x_i = w_i$ trivially constant (Theorem 5.10). Without discreteness, the NAU would be a general linear layer with no structural advantage over standard networks. In the formalization, this is encoded as:

```
structure NAULayer (n : ℕ) where
weights : Fin n → ℤ
weight_discrete : ∀ i, weights i = -1 ∨ weights i = 0 ∨ weights i = 1
```

J.2 SINDy Coefficients and Reconstruction

Definition J.2 (SINDy Coefficients)**.** *Given a basis library $\{\varphi_1, \ldots, \varphi_n\}$, SINDy coefficients are a vector $\xi \in \mathbb{R}^n$ defining the symbolic formula $f_{sym}(x) = \sum_{i=1}^{n} \xi_i \varphi_i(x)$.*

The `reconstruct` function maps coefficients back to the function space, enabling direct comparison with π and f_{real}. The Fréchet derivative of the reconstruction is $Df_{\text{sym}}(x) = \sum_i \xi_i D\varphi_i(x)$, which is used in the Jacobian Consistency definition (Definition 5.7). In formal notation:

```
structure SINDyCoeffs (n : ℕ) where
  vals : Fin n → ℝ

def SINDyCoeffs.reconstruct
  (c : SINDyCoeffs n) (lib : BasisLibrary n E F) : E → F :=
  fun x ⇒ ∑ i, c.vals i • lib.basis i x
```

J.3 IRM Environment

Definition J.3 (Environment)**.** *An environment consists of a training domain $D_{train} \subseteq E$ and a ground-truth response function $f_{real} : E \to F$.*

IRM filtering (Pillar III) requires comparing symbolic formulas across *multiple* environments. Each environment shares the same state/action spaces but has distinct dynamics parameters (e.g., different masses, friction coefficients). The IRM Consistency predicate (Definition 5.20) requires that a single SINDy coefficient vector ξ achieves low derivative error *simultaneously* across all environments in the list, ensuring causal rather than spurious correlation. In formal notation:

```
structure Environment (E F : Type*)
[NormedAddCommGroup E] [NormedSpace ℝ E]
[NormedAddCommGroup F] [NormedSpace ℝ F] where
D_train : Set E
f_real : E → F
```

J.4 NMU Unit

Definition J.4 (NMU Unit)**.** *An NMU unit with coefficient $c \in \mathbb{R}$ computes $f(x) = cx^2$.*

The single-parameter design is deliberate: the NMU's second derivative is $f''(x) = 2c$ (constant), yielding the exact Lipschitz constant $L = 2|c|$ for the first derivative (Theorem 5.11). Richer polynomial units (e.g., $f(x) = c_1 x^2 + c_2 x^3$) would have *non-constant* second derivatives, making the Lipschitz constant depend on the input domain and losing the global guarantee. In formal notation:

```
structure NMUUnit where
coeff :  ℝ
```